SkillBridge : Ai-Based Skill Gap Analyzer and Roadmap Generator

Submitted in partial fulfilment of the requirement for the award of the degree of

Master of Computer Application (MCA)

Tecnia Institute of Advanced Studies

Affiliated to

Guru Gobind Singh Indraprastha University,

Delhi

Supervisor:

Guide – Mr. Akshit Bhardwaj

Assistant Professor

Co-Guide – Mr. Deepak Rathore

Assistant Professor

Submitted by:

Shubham Bhandari

00517004425

MCA 2nd Sem.

Batch (2025-2027)

MCA – 170

TECNIA INSTITUTE OF ADVANCED STUDIES

NAAC GRADE "A" INSTITUTE (CYCLE-2)

Approved by AICTE, Ministry of Education Govt. of India,

Affiliated to G.G.S.I.P. University & Recognized Under Sec. 2(f) of UGC Act 1956.

INSTITUTIONAL AREA, MADHUBAN CHOWK, ROHINI, DELHI-110085

CERTIFICATE

This is to certify that this project entitled **SkillBridge : Ai-Based Skill Gap Analyzer and Roadmap Generator** submitted in partial fulfilment of the degree of Master of Computer Applications to the "**Mr. Akshit and Mr. Deepak Rathore**" through the ICT department of Tecnia Institute of Advanced Studies by Shubham Bhandari**,** Roll No 00517004425 is an authentic work carried out by him/her under my guidance. The matter embodied in this project work has not been submitted earlier for the award of any degree to the best of my knowledge and belief.

Signature of the student

Date:

Certified that the Project Report/Dissertation (MCA 170) entitled **"SkillBridge : Ai-Based Skill Gap Analyzer and Roadmap Generator"** done by Mr. Shubham Bhandari, Roll No. 00517004425 is completed under my guidance.

Countersigned

Director/HOD

Guide: Mr. Akshit

(Assistant Professor)

Signature:

Co-Guide: Mr. Deepak Rathore

(Assistant Professor)

Signature:

Address: Tecnia Institute of Advanced Studies, Madhuban Chowk, New Delhi-110085

ACKNOWLEDGEMENT

This certifies that **Shubham Bhandari**, Roll No. **00517004425**, student at **Tecnia Institute of Advance Studies**, has successfully completed the project **SkillBridge : Ai-Based Skill Gap Analyzer and Roadmap Generator** under my supervision during **2026**. I would like to express my heartfelt gratitude to all those who have contributed to the successful completion.

(Signature)

Student Name: Shubham Bhandari

Student Enrolment No: 00517004425

PREFACE

The rapid advancement of technology has fundamentally transformed the nature of work, creating an ever-widening gap between the skills individuals possess and those demanded by the modern job market. This project, **SkillBridge**: **AI-Based Skill Gap Analyzer and Roadmap Generator**, was conceived with the intent to address this challenge through the intelligent application of artificial intelligence and full-stack web development.

This report documents the complete lifecycle of the project — from initial conceptualization and requirement analysis through system design, development, testing, and implementation. It has been developed as part of the Minor Project (MCA-170) curriculum of the Master of Computer Applications program at Tecnia Institute of Advanced Studies, affiliated to Guru Gobind Singh Indraprastha University, Delhi, for the academic year 2024-25.

The system presented in this report goes beyond conventional skill assessment tools by combining user-declared skills with objective MCQ-based competency testing and AI-generated learning roadmaps. The core innovation — the Intelligent Skill Merging Engine — synthesizes multiple data signals to produce a nuanced, five-category skill profile that provides a far more accurate picture of a user's true competency landscape than self-assessment alone can offer.

This report is intended for academic evaluation and is organized in accordance with the project report guidelines prescribed by the Department of Information Communication and Technology, Tecnia Institute of Advanced Studies. It is hoped that this work will serve as a useful reference for future students undertaking similar projects at the intersection of artificial intelligence and web application development.

SYNOPSIS

SKILLBRIDGE: AI-BASED SKILL GAP ANALYZER AND ROADMAP GENERATOR

1. Abstract

The accelerating pace of technological change has created a persistent and widening gap between the skills job seekers possess and those demanded by the modern workforce. Traditional self-assessment methods are inherently subjective and unreliable, as individuals frequently overestimate or underestimate their own competencies. This project presents an AI-Based Skill Gap Analyzer and Roadmap Generator — a full-stack web application that employs an intelligent, multi-layered approach to skill evaluation and personalized career guidance.

Built on a three-tier architecture comprising a React (Vite) frontend, Node.js/Express.js REST API backend, and MongoDB database, the system integrates the Groq API with the LLaMA 3.3-70B model to generate prioritized, structured learning roadmaps. The platform features a multi-step user flow with authentication, optional resume upload, role-specific MCQ assessment, an interactive analytics dashboard, and a projected score simulation module. This project demonstrates the effective integration of AI-driven assessment, generative AI, and modern web technologies to deliver objective, actionable career development guidance.

2. Introduction

In today's rapidly evolving technology landscape, the concept of a skill gap — the disparity between competencies an individual possesses and those required for a specific role — has become one of the most significant challenges facing both job seekers and employers. The World Economic Forum estimates that over half of all employees will require significant reskilling by 2025, underscoring the urgency of accessible, intelligent career guidance tools.

Conventional approaches to skill self-assessment are fundamentally flawed. When individuals evaluate their own skills through resume declarations or subjective ratings, cognitive biases such as the Dunning-Kruger effect introduce systematic inaccuracies. Individuals with limited knowledge of a domain often overestimate their proficiency,

while experts may underestimate theirs. Furthermore, self-reported skills on resumes and job applications are rarely validated against objective performance criteria, resulting in misaligned hiring decisions and inefficient learning investments.

This project addresses these challenges by designing a platform that combines role-specific MCQ-based competency testing with AI-generated roadmaps. The system's Intelligent Skill Merging Engine objectively reconciles self-declared skills with test performance, detecting hidden competencies users may not have declared (Discovered skills) and clearly identifying Weak areas requiring immediate attention. The Groq API, powered by the LLaMA 3.3-70B model, generates learning roadmaps that prioritize Weak skills before Missing ones — ensuring the most impactful learning investments are surfaced first.

3. Problem Statement

Despite the availability of numerous online learning platforms, career portals, and job matching services, a critical gap persists in tools capable of objectively assessing a user's skill profile and generating actionable, prioritized learning guidance. The primary challenges motivating this project are:

- Inaccuracy of Self-Declared Skills: Resume-based skill declarations are inherently unvalidated. Users list skills without standardized proficiency benchmarks, making it impossible for systems relying solely on self-reported data to produce accurate gap analyses.
- Absence of Objective Competency Measurement: Most existing skill gap tools lack an integrated assessment mechanism. Without objective testing, the system cannot distinguish between surface-level familiarity and genuine proficiency in a skill area.
- Undiscovered Competencies: Users frequently fail to declare skills they possess but do not associate with their professional identity. Without a testing component, such hidden talents remain undetected, potentially leading to underqualification perceptions.
- Generic Learning Recommendations: Existing platforms tend to offer static, non-prioritized learning paths that do not account for the relative urgency of

different skill gaps. A user with Weak skills in core competencies needs different guidance than one with entirely Missing skills.

This project fills these gaps by combining objective MCQ-based assessment with an Intelligent Skill Merging Engine and AI-generated, prioritized roadmaps — delivering a single, integrated platform for honest, data-driven career development.

4. Objectives and Scope of Work

4.1 Objectives

- To develop a secure, full-stack web application with user authentication (login/signup) enabling personalized skill gap analysis.
- To implement a multi-step user flow: Authentication -> Name/Email -> Resume Upload (optional) -> Target Role Selection -> MCQ Skill Assessment -> AI Analysis Dashboard.
- To build a role-specific MCQ Question Bank with each question tagged to a specific skill, enabling per-skill proficiency calculation.
- To develop an Intelligent Skill Merging Engine that synthesizes user-declared skills and test results into a Final Skill Profile with Verified, Discovered, Declared, Weak, and Missing classifications.

4.2 Scope of Work

- User registration, login, and session management with secure authentication.
- Multi-step onboarding flow with resume upload and role selection.
- Role-specific MCQ assessment with server-side answer evaluation and per-skill proficiency scoring.
- Intelligent Skill Merging Engine with five-category skill classification.
- Groq AI-powered roadmap generation with fallback roadmap for API unavailability.
- Interactive dashboard with animated circular progress bars, skill breakdown, career level indicator, and certification recommendations.

5. Literature Review and Research Gap

Several studies have explored skill gap analysis, learning recommendations, and assessment systems:

- **Khanal et al. (2020):**
  Highlighted that hybrid recommendation systems improve relevance in e-learning. This supports the roadmap generation approach used in this project.
- **Chatti et al. (2012):**
  Proposed a learning analytics model focused on data collection, analysis, and actionable insights. This influences the skill evaluation and merging logic.
- **Xu et al. (2016):**
  Showed that personalized learning paths based on user competency improve learning outcomes. This validates the prioritized roadmap design.

Research Gap:

Existing work addresses recommendation systems, analytics, or assessment separately. However, no unified system combines **skill validation (MCQs), skill gap detection, AI-based roadmap generation, and progress prediction** in a single platform.

This project fills that gap by integrating all components into one end-to-end system.

6. Proposed System Design

6.1 System Architecture Overview

The system follows a **three-tier client-server architecture**:

- **Presentation Layer (Frontend):**
  Built using React.js (Vite), handling authentication, onboarding, assessment, and dashboard. Axios manages API calls, with UI components like progress bars, skill cards, and roadmap tracking.
- **Application Layer (Backend):**
  Node.js/Express server providing REST APIs. Handles authentication (JWT), resume parsing (multer, pdf-parse), assessment evaluation, skill merging logic, and AI roadmap generation via Groq API.

- **Data Layer (Database):**
  MongoDB stores data across Users, JobRoles, QuestionBank, and LearningPaths collections. Managed using Mongoose.

6.2 Data Flow Diagram (DFD)

Level 0 — Context Diagram:

The Level 0 DFD represents the entire system as a single process with three external entities:

- **User → System:** Inputs credentials, resume, role selection, and answers.
- **System → User:** Outputs skill profile, scores, roadmap, and insights.
- **System ↔ Groq API:** Sends skill data and receives AI-generated roadmap..

Level 1 — Detailed DFD:

The Level 1 DFD decomposes the system into six primary processes:

1. Process 1.0 — User Authentication: Accepts registration/login credentials. Validates against the Users collection.
2. Process 2.0 — User Onboarding: Collects name, email, current skills, and optional PDF resume.
3. Process 3.0 — MCQ Skill Assessment: Retrieves role-specific questions from the QuestionBank collection (questions served without answers).

6.3 Entity-Relationship Diagram (ERD)

Entities:

- **User:** id, name, email, passwordHash
- **JobRole:** id, title, requiredSkills, description
- **QuestionBank:** id, role, question, options
- **LearningPath:** id, userId, targetRole, finalSkillProfile

Relationships:

- One **User → multiple LearningPaths**
- One **JobRole → multiple Questions & LearningPaths**
- Each **Question → one JobRole**

7. Tools and Technologies Used

Technology / Tool	Category	Purpose
React.js (Vite)	Frontend Framework	Building the interactive multi-step UI; Vite provides fast HMR and build tooling.
Axios	HTTP Client	Asynchronous API communication between frontend and backend.
react-circular-progressbar	UI Component	Animated circular progress bars for Match Score and Projected Score display.
Node.js	Backend Runtime	Server-side JavaScript execution environment.
Express.js	Backend Framework	RESTful API routing, middleware management, and authentication handling.
MongoDB	NoSQL Database	Storing users, job role profiles, question bank, and learning paths.
Mongoose	ODM Library	Schema definition, validation, and MongoDB query abstraction.
Groq API (LLaMA 3.3-70B)	AI / LLM Service	Generating personalized, prioritized learning roadmaps from Final Skill Profile data.
VS Code	IDE	Primary development environment.
Postman	API Testing	Testing and validating all REST API endpoints.
Git / GitHub	Version Control	Source code management and collaboration.

8. Implementation and Methodology

The project follows an **Agile SDLC approach** with iterative development phases:

- **Requirement Analysis:**
  Define system features such as onboarding, resume parsing, skill assessment, and roadmap generation based on user needs.

- **Design Phase:**
  Design system architecture, MongoDB schemas (User, JobRole, QuestionBank, LearningPath), REST APIs, and React component structure.
- **Development:**
  Build backend (Node.js, Express) and frontend (React). Implement authentication (JWT, bcrypt), resume parsing, assessment engine, skill analysis, and AI roadmap generation using Groq API.
- **Integration:**
  Connect frontend, backend, database (MongoDB), and AI services through REST APIs.
- **Testing:**
  Verify functionality of authentication, assessment, skill classification, and roadmap generation.
- **Deployment:**
  Run locally or deploy on a server with proper configuration.
- **Maintenance:**
  Fix bugs, improve performance, and update features based on feedback.

ALGORITHMIC FLOW:

- User registers/logs in and enters personal details.
- User uploads resume (PDF), which is parsed to extract skills.
- User selects a target job role.
- The system fetches role-specific questions from the database.
- User completes the assessment through the web interface.
- Backend evaluates answers and calculates skill-wise performance.
- The system classifies skills into categories (Strong, Moderate, Weak).

9. Testing Plan, Results and Discussion

A multi-layered testing strategy was applied across unit, integration, and UI levels to ensure system correctness, reliability, and robustness.

9.1 Unit Testing

- Intelligent Skill Merging Engine: Tested with multiple combinations of declared skills and test proficiency inputs to verify correct classification into all five categories (Verified, Discovered, Declared, Weak, Missing). Edge cases tested include: skills with no test data, skills with borderline scores (exactly 40% and 50%), and roles with no matching question bank entries.
- Per-Skill Proficiency Calculation: Verified that (correct / total) * 100 produces accurate results across all skill groups, including single-question skills and skills with all correct or all incorrect responses.

9.2 Integration Testing

- Full data flow tested: Frontend form submission -> /api/analyze endpoint -> Skill Merging Engine -> Groq API -> LearningPaths collection -> Dashboard response rendering.
- Authentication flow tested: Registration -> login -> JWT issuance -> protected route access -> token expiry handling.

9.3 API Testing (Postman)

- All REST endpoints tested with valid, invalid, and edge-case payloads including: missing required fields, invalid JWT tokens, unrecognized role names, empty skill arrays, and malformed assessment submissions.
- HTTP status codes (200, 201, 400, 401, 404, 500) and error response schemas verified against API specifications.

9.4 UI Testing

- Multi-step flow tested for correct state persistence across steps and prevention of step-skipping without completing mandatory fields.
- Animated circular progress bars verified to render correct percentages for Match Score and Projected Score.
-

9.5 Expected Results

Upon successful deployment, the system is expected to: accurately classify skills into all five categories with zero misclassification for test cases within defined boundary conditions; generate relevant, prioritized roadmaps with Weak skills consistently

ranked before Missing skills; produce Match Scores and Test Scores that are logically independent yet collectively informative; and provide a Projected Score that reflects actual task completion rather than a static estimate.

10. Expected Outcomes and Future Work

10.1 Expected Outcomes

- A production-ready web application providing objective, multi-signal skill gap analysis combining self-declaration, MCQ assessment, and AI-generated insights.
- Accurate detection of Discovered skills — competencies users possess but did not self-report — addressing a gap unaddressed by self-assessment-only platforms.
- Prioritized, actionable learning roadmaps that direct users toward the highest-impact skill development activities first.
- A Projected Score Simulation module that quantifies the benefit of completing learning tasks, increasing user motivation and goal clarity.
- A scalable, modular architecture ready for production deployment and iterative feature addition.

10.2 Future Work

- Semantic Resume Parsing: Integrate NLP-based entity recognition (e.g., spaCy or Hugging Face models) to automatically extract and normalize skill entities from resume text, replacing the current plain-text approach.
- LinkedIn Profile Integration: Connect with LinkedIn's API to import professional experience and endorsements, enriching the skill declaration baseline with externally validated data.
- Progress Tracking Over Time: Implement a longitudinal tracking module allowing users to retake assessments at intervals and visualize their skill development trajectory over weeks and months.

11. References

[1] Khanal, S. S., Prasad, P. W. C., Alsadoon, A., & Maag, A. (2020). A systematic review: machine learning based recommendation systems for e-learning. Education and Information Technologies, 25, 2635-2664. https://doi.org/10.1007/s10639-019-10063-9

[2] Chatti, M. A., Dyckhoff, A. L., Schroeder, U., & Thus, H. (2012). A reference model for learning analytics. International Journal of Technology Enhanced Learning, 4(5-6), 318-331. https://doi.org/10.1504/IJTEL.2012.051647

[3] Xu, J., Xing, T., & Van der Schaar, M. (2016). Personalized course sequence recommendations. IEEE Transactions on Signal Processing, 64(20), 5340-5352. https://doi.org/10.1109/TSP.2016.2591500

[4] Aher, S. B., & Lobo, L. M. R. J. (2013). Combination of machine learning algorithms for recommendation of courses in E-Learning System based on historical data. Educational Data Mining, 5(4), 37-55.

[5] Imran, A. S., Dalipi, F., & Folke, M. (2021). AI-Powered Adaptive Learning Systems. IEEE Access, 9, 98345-98360. https://doi.org/10.1109/ACCESS.2021.3095650

[6] Romero, C., & Ventura, S. (2020). Educational data mining and learning analytics: An updated survey. WIREs Data Mining and Knowledge Discovery, 10(3), e1355. https://doi.org/10.1002/widm.1355

[7] Baker, R. S., & Inventado, P. S. (2014). Educational data mining and learning analytics. In Larusson J., White B. (eds), Learning Analytics. Springer. https://doi.org/10.1007/978-1-4614-3305-7_2

[8] Fazeli, S., Drachsler, H., & Sloep, P. (2017). Applying recommender systems for learning analytics: A tutorial and review. New Directions for Teaching and Learning, 2017(152), 105-124. https://doi.org/10.1002/tl.20273

[9] Verbert, K., Manouselis, N., Ochoa, X., Wolpers, M., Drachsler, H., Bosnic, I., & Duval, E. (2012). Context-aware recommender systems for learning: A survey and future challenges. IEEE Transactions on Learning Technologies, 5(4), 318-335. https://doi.org/10.1109/TLT.2012.11

[10] Dillahunt, T. R., Wang, X., & Teasley, S. D. (2016). Democratizing Higher Education: Exploring MOOC Use Among Those Who Cannot Afford a College Education. International Review of Research in Open and Distributed Learning, 17(6). https://doi.org/10.19173/irrodl.v17i6.2462

[11] Groq Inc. (2024). Groq API Documentation. Retrieved from https://console.groq.com/docs

[12] Meta AI. (2024). LLaMA 3.3 Model Card. Retrieved from https://ai.meta.com/llama

[13] MongoDB Inc. (2024). MongoDB Documentation: Schema Design Best Practices. Retrieved from https://www.mongodb.com/docs

[14] Facebook Inc. (2024). React Documentation. Retrieved from https://react.dev

[15] Node.js Foundation. (2024). Node.js Documentation. Retrieved from https://nodejs.org/en/docs

[16] Express.js Contributors. (2024). Express Documentation. Retrieved from https://expressjs.com

[17] Mongoose Contributors. (2024). Mongoose ODM Documentation. Retrieved from https://mongoosejs.com/docs

INDEX

S No.	Topic	Page No.
1	Certificate(s)	II
2	Acknowledgement	III
3	Preface	IV
4	Synopsis	V
5	**Chapter-1: Introduction**	**1-5**
	1: Brief Description	1
	2: Background	1
	3: User Requirements	2
	4: Problem Definition	2
	5: Objective of the Project	3
	6: Methodology	4
	7: Data Required and Collection	5
6	**Chapter-2: System Requirement Analysis**	**6-7**
	1: Analysis of the Existing System	6
	2: Identification of Processes	7
	3: Input and Output Identification	8
	4: Functional Requirements	9
	5: Non-Functional Requirements	10
7	**Chapter-3: System Design**	**12-15**
	1: System Architecture	12
	2: Data Flow Diagrams	13
	3: Entity-Relationship Diagram	17
	4: Use Case Diagram	20
	5: Multi-Step User Flow	21
	6: Database and File Design	22
	7: Intelligent Skill Merging Engine	25
8	**Chapter-4: System Development**	**27-42**
	1: Programming Language and Platform	27
	2: Development Methodology	27
	3: REST API Endpoint Reference	30

	4: Tools and Technologies	31
	5: Query Language	32
	6: Output	41
9	**Chapter-5: Systems Implementation**	**46-52**
	1: Testing Strategy	46
	2: Test Plan	47
	3: Types of Testing	48
	4: Test Cases	50
	5: Verification and Validation	50
	6: Test Environment	51
	7: Test Summary and Implementation	53
9	**Chapter-6: Systems Implementation**	**53-56**
	1: System Requirements — Acquisition	53
	2: Changeover Procedure	54
	3: Training Needs	54
	4: Documentation	55
10	Summary/Conclusion	57
11	Limitations Of the Project	58
12	Future Directions	59
13	References / Bibliography	60

LIST OF TABLES

Table No.	Table Title	Page No.
Table No. 1	Functional Requirements	9
Table No. 2	Non-Functional Requirements	10
Table No. 3	Skill Classification Rules — Intelligent Merging Engine	10
Table No. 4	User Collection — Database Field Design	22
Table No. 5	JobRole Collection — Database Field Design	23
Table No. 6	QuestionBank Collection — Database Field Design	23
Table No. 7	LearningPath Collection — Database Field Design	23
Table No. 8	REST API Endpoint Reference	30
Table No. 9	Tools and Technologies Used	31
Table No. 10	Unit Test Cases — Skill Merging Engine	50
Table No. 11	System Hardware and Software Requirements	53

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure No-1	System Architecture — Three-Tier Overview	12
Figure No-2	DFD Level 0 — Context Diagram	14
Figure No-3	DFD Level 1 — Detailed Data Flow	15
Figure No-4	Entity-Relationship Diagram (ERD)	17
Figure No-5	Use Case Diagram	20
Figure No-6	Multi-Step User Flow Diagram	21
Figure No-7	Intelligent Skill Merging Engine — Logic Flowchart	25
Figure No-8	Agile Sprint Timeline	27
Figure No-9	Login and Registration Page	44
Figure No-10	User Onboarding Form	45
Figure No-11	Role Selection Page	45
Figure No-12	MCQ Assessment Screen	46
Figure No-13	Analytics Dashboard — Score and Career Level	47
Figure No-14	Skill Breakdown Section	47
Figure No-15	Learning Roadmap Section	48
Figure No-16	Projected Score Modal	48

LIST OF SYMBOLS

S.No.	Symbol	Nomenclature and Meaning
1	API	Application Programming Interface
2	REST	Representational State Transfer
3	HTTP	Hypertext Transfer Protocol
4	JWT	JSON Web Token
5	MCQ	Multiple Choice Question
6	AI	Artificial Intelligence
7	LLM	Large Language Model
8	NLP	Natural Language Processing
9	DFD	Data Flow Diagram
10	ERD	Entity-Relationship Diagram
11	SDLC	Software Development Life Cycle
12	ODM	Object Document Mapper
13	PDF	Portable Document Format
14	UI	User Interface
15	UX	User Experience
16	CRUD	Create, Read, Update, Delete
17	HMR	Hot Module Replacement
18	MCA	Master of Computer Applications
19	%	Percentage
20	>=	Greater than or equal to

CHAPTER-1

INTRODUCTION

1 Brief Description

This project has been developed as an in-house academic project at Tecnia Institute of Advanced Studies, Madhuban Chowk, Rohini, New Delhi-110085, an institution affiliated to Guru Gobind Singh Indraprastha University (GGSIPU), Delhi. The institute is an NAAC Grade 'A' accredited institution approved by AICTE and recognized under Section 2(f) of the UGC Act 1956. The Department of Information Communication and Technology (DICT) offers programs in MCA, BCA, and B.Tech., emphasizing practical skill development and technology-driven project work.

The project was conceptualized and developed to address a real-world problem in the career development domain, specifically the inability of existing tools to objectively measure and bridge the gap between job seekers' current skills and the requirements of their target roles.

2 Background

The existing approach to skill gap assessment and career guidance predominantly relies on self-reported information. Job seekers manually compare their resume against job descriptions — a process that is inherently subjective, inconsistent, and cognitively biased. Professionals on platforms such as LinkedIn self-rate their skills on arbitrary scales without any standardized validation mechanism. Career counsellors rely on interviews and subjective assessments, which are time-consuming and inaccessible to many.

Online learning platforms such as Coursera, Udemy, and LinkedIn Learning offer course recommendations, but these are typically driven by browsing behaviour and broad interest

categories rather than an objective, verified assessment of the user's actual competency level for a specific target role. No widely available integrated platform combines objective testing, multi-signal skill merging, and AI-generated prioritized roadmaps in a single workflow.

3 User Requirements

The target users of this system are students and early-career professionals who are preparing to enter or transition within the technology industry. Based on requirements analysis, the following user needs were identified:

- Users need an objective, unbiased method to assess their actual skill levels for a specific target job role, independent of self-perception biases.
- Users need to discover competencies they possess but may not have formally recognized or declared on their resume.
- Users need a clear, structured, and prioritized learning roadmap that directs them toward the most impactful skill development activities first.
- Users need visibility into their match score against a target role, with a mechanism to simulate how completing learning tasks improves that score.
- Users need secure, personalized access to their skill analysis results and learning progress.
- Users need a seamless, guided experience from initial profile creation through assessment to dashboard without technical friction.

4 Establishing the Need for the New System

4.1 Justification for the Proposed System

The limitations of the existing system create a clear and urgent need for an intelligent, automated alternative. Self-reported skills cannot be trusted as the sole input for gap analysis

— a user who lists 'JavaScript' on their resume may have anywhere from beginner to expert-level proficiency. Without objective validation, any gap analysis built on such data is fundamentally unreliable.

Furthermore, the absence of automated testing means that Discovered skills — competencies a user possesses but did not think to declare — remain invisible to the system. A comprehensive skill profile must account for both what users say they know and what they can demonstrate through performance.

4.2 How the Proposed System Addresses These Gaps

The proposed AI-Based Skill Gap Analyzer and Roadmap Generator resolves these deficiencies through the following mechanisms:

- Role-specific MCQ assessment provides an objective, standardized measure of skill proficiency that complements self-declared skills.
- The Intelligent Skill Merging Engine synthesizes declared skills and test results into a five-category Final Skill Profile, eliminating reliance on any single data source.
- The Groq API (LLaMA 3.3-70B) generates personalized, prioritized learning roadmaps based on the actual Final Skill Profile, ensuring recommendations are tailored and actionable.
- The Projected Score Simulation quantifies the impact of completing learning tasks, providing users with a concrete, motivating goal.
- JWT-based authentication ensures data security and personalized session management.

5 Objectives of the Project

The following objectives were defined for the project:

1. To develop a secure full-stack web application with user authentication enabling personalized, persistent skill gap analysis.

2. To implement a multi-step user onboarding flow: Authentication, Name/Email and Resume Upload, Target Role Selection, MCQ Skill Assessment, and AI Analysis Dashboard.
3. To build a role-specific MCQ Question Bank with each question tagged to a specific skill for per-skill proficiency calculation.
4. To develop an Intelligent Skill Merging Engine that synthesizes declared skills and test performance into a Verified, Discovered, Declared, Weak, and Missing skill profile.
5. To compute a Match Score (skill coverage) and Test Score (skill depth) as independent, non-contradicting metrics.
6. To integrate the Groq API (LLaMA 3.3-70B) for AI-generated, prioritized learning roadmaps with Weak skills ranked before Missing skills.
7. To develop an interactive analytics dashboard with eleven sections including animated progress visualization, career level indicator, and certification recommendations.
8. To implement a Projected Score Simulation module enabling users to visualize the impact of task completion on their match score.

6 Methodology

The project follows the Agile Software Development Life Cycle (SDLC) model, which supports iterative development, continuous feedback, and incremental delivery. Agile was chosen over the traditional Waterfall model due to the evolving nature of requirements — particularly the addition of the authentication module, resume parsing, and the Projected Score feature during development.

The development is structured across eight sprints, each producing a functional, testable increment of the system. Structured analysis and design tools including DFDs, ERDs, and Use

Case Diagrams were used to model the system before implementation. Testing was conducted at unit, integration, and system levels throughout the development process.

7 Data Required and Data Collection Method

The following categories of data are required by the system:

- User Data: Name, email, password, declared current skills, target job role, and optional resume text — collected through the multi-step onboarding form.
- Job Role Profiles: Required skills for each target role (e.g., Full Stack Developer, Data Scientist, DevOps Engineer) — created and seeded into the MongoDB JobRole collection during system setup.
- Question Bank Data: Role-specific MCQ questions with answer options, correct answers, and skill tags — authored and seeded into the QuestionBank collection.
- Assessment Responses: User's MCQ answers, submitted through the assessment interface and evaluated server-side.
- Resume Text: Optional — extracted from uploaded PDF files using the pdf-parse library.

Data collection for job role profiles and question bank content was conducted through analysis of industry job descriptions, technical interview repositories, and standard competency frameworks for software development roles.

CHAPTER-2

SYSTEM REQUIREMENT ANALYSIS

1 Analysis of the Existing System

1.1 Existing System and Its Limitations

The current landscape of skill assessment and career guidance tools can be categorized into three types: self-assessment platforms, course recommendation engines, and manual career counselling services. Each suffers from distinct limitations:

- Self-Assessment Platforms (e.g., LinkedIn Skills): Rely entirely on user self-rating without any objective validation. Proficiency levels are subjective and cannot be compared across users. The Dunning-Kruger effect causes systematic inaccuracy — novices frequently overestimate their skills while experts may underestimate theirs.
- Course Recommendation Engines (e.g., Coursera, Udemy): Recommend learning content based on browsing history, interests, and broad categories. They do not assess the user's existing proficiency level for a specific target role, resulting in recommendations that may be redundant (too easy), overwhelming (too advanced), or simply misaligned with actual gaps.
- Manual Career Counselling: Provides personalized guidance but is expensive, time-consuming, geographically limited, and not scalable. Counsellor assessments are subjective and vary in quality.
- Resume Parsing Tools: Extract skill keywords from resumes but treat all declared skills as equally valid, making no distinction between strong, moderate, and weak proficiency.

1.2 Proposed System and Its Objectives

The proposed system supersedes these limitations by combining three data signals — self-declaration, objective testing, and AI analysis — into a unified, automated platform. The specific improvements over the existing system are:

- Objective skill validation through role-specific MCQ assessment eliminates the subjectivity of self-rating.
- The Intelligent Skill Merging Engine produces a nuanced, five-category skill profile rather than a binary declared/not-declared classification.
- AI-generated roadmaps are personalized to the user's actual Final Skill Profile and prioritized by urgency (Weak before Missing).
- The Projected Score Simulation provides actionable, quantified motivation for learning task completion.

1.3 Feasibility Study

Technical Feasibility: All technologies selected (React, Node.js, MongoDB, Groq API) are mature, well-documented, and freely available. The Groq API provides free-tier access to the LLaMA 3.3-70B model with sufficient rate limits for academic use. The system can be deployed on any standard web hosting infrastructure.

Operational Feasibility: The system is designed with a clean, guided multi-step interface requiring no technical expertise from users. The onboarding flow is self-explanatory and the dashboard presents complex analysis data in an accessible, visual format.

Economic Feasibility: The project uses entirely open-source technologies with no licensing costs. The Groq API free tier is sufficient for development and academic demonstration. Operational costs for deployment are minimal.

2 Identification of Processes

The following primary processes have been identified in the proposed system:

- User Authentication Process: Handles registration, login, JWT token issuance, and session management.
- User Onboarding Process: Collects name, email, current skills, and optional resume. Extracts resume text via pdf-parse.
- Role Selection Process: Presents available job role profiles. User selects target role. System fetches role's required skills from MongoDB.
- MCQ Assessment Process: Serves role-specific questions without answers. Collects user responses. Evaluates answers server-side. Computes per-skill proficiency.
- Skill Merging Process: Merges declared skills and test results. Applies classification rules. Computes Match Score and Career Level.
- AI Roadmap Generation Process: Constructs prompt from Final Skill Profile. Calls Groq API. Parses and stores roadmap. Activates fallback if API unavailable.
- Dashboard Delivery Process: Compiles all analysis data. Returns structured response to frontend. Renders eleven dashboard sections.
- Projected Score Process: Tracks task completion. Calculates projected score on full completion. Returns result for modal display.

3 Input and Output Identification

3.1 Inputs

- Authentication: Email address, password (registration adds name field).
- Onboarding: Full name, email confirmation, current skills (comma-separated or tag input), optional PDF resume file.

- Role Selection: Selected job role from dropdown list.
- MCQ Assessment: Array of selected answer options (one per question, submitted as a batch).
- Task Tracking: Checkbox state (completed/incomplete) for each roadmap task.

3.2 Outputs

- Authentication: JWT access token; user session data.
- Skill Analysis: Final Skill Profile (categorized skills), Match Score (percentage), Test Score (percentage), Career Level label.
- Dashboard: AI Insight Summary, Matched Skills list, Required Skills Coverage grid, Skill Breakdown cards, AI Skill Insights sentences, Skill Assessment performance bars, Skills to Develop list, Learning Roadmap with tasks, Certification Recommendations.
- Projected Score: Current score, projected score, score gain value — rendered in animated modal.

4 Functional Requirements

FR No.	Feature	Description
FR-01	User Registration	System shall allow new users to register with name, email, and password. Password shall be stored as bcrypt hash.
FR-02	User Login	System shall authenticate registered users and issue a JWT token valid for the session duration.
FR-03	Resume Upload	System shall accept PDF resume uploads via multer and extract plain text via pdf-parse.
FR-04	Role Selection	System shall present all available job role profiles and allow user to select one target role.
FR-05	MCQ Assessment	System shall serve role-specific questions without answers and evaluate submitted answers server-side.

FR-06	Skill Merging	System shall merge declared skills and test results into Verified, Discovered, Declared, Weak, Missing categories.
FR-07	Match Score	System shall compute Match Score as (Verified + Discovered + Declared) / total required skills * 100.
FR-08	AI Roadmap	System shall generate a prioritized learning roadmap via Groq API with fallback logic.
FR-09	Dashboard	System shall render all eleven dashboard sections with animated visualizations.
FR-10	Projected Score	System shall calculate and display projected score in animated modal after full task completion.

Table No. 1: Functional Requirements

5 Non-Functional Requirements

NFR No.	Category	Requirement
NFR-01	Performance	API responses (excluding AI generation) shall complete within 2 seconds. AI roadmap generation shall complete within 8 seconds.
NFR-02	Security	All passwords shall be bcrypt hashed. All protected endpoints shall require valid JWT. Answer keys shall never be transmitted to the frontend.
NFR-03	Reliability	System shall activate fallback roadmap generation if Groq API is unavailable, ensuring zero system failure due to external API downtime.
NFR-04	Usability	Multi-step flow shall prevent step-skipping without completing mandatory fields. All dashboard sections shall be clearly labeled.
NFR-05	Scalability	MongoDB schema design and modular Express routing shall support addition of new job roles and question bank entries without code changes.
NFR-06	Maintainability	Codebase shall follow modular architecture with separate route, controller, and model files for each domain entity.

Table No. 2: Non-Functional Requirements

6 Skill Classification Rules

Category	Declared?	Test Score	Description
Verified	Yes	>= 50%	User declared and demonstrated proficiency. Counts in Match Score.
Discovered	No	>= 50%	Hidden talent detected through testing. Counts in Match Score.
Declared	Yes	No data	Declared but not tested. Counts in Match Score.
Weak	Any	< 40%	Test reveals low proficiency. Does not count in Match Score. Prioritized first in roadmap.
Missing	No	< 50% or none	Not declared and not demonstrated. Does not count. Second priority in roadmap.

Table No. 3: Skill Classification Rules — Intelligent Merging Engine

CHAPTER-3

SYSTEM DESIGN

1 System Architecture

The system is designed as a three-tier client-server architecture. This separation of concerns ensures maintainability, scalability, and clear delineation of responsibilities across the presentation, logic, and data layers.

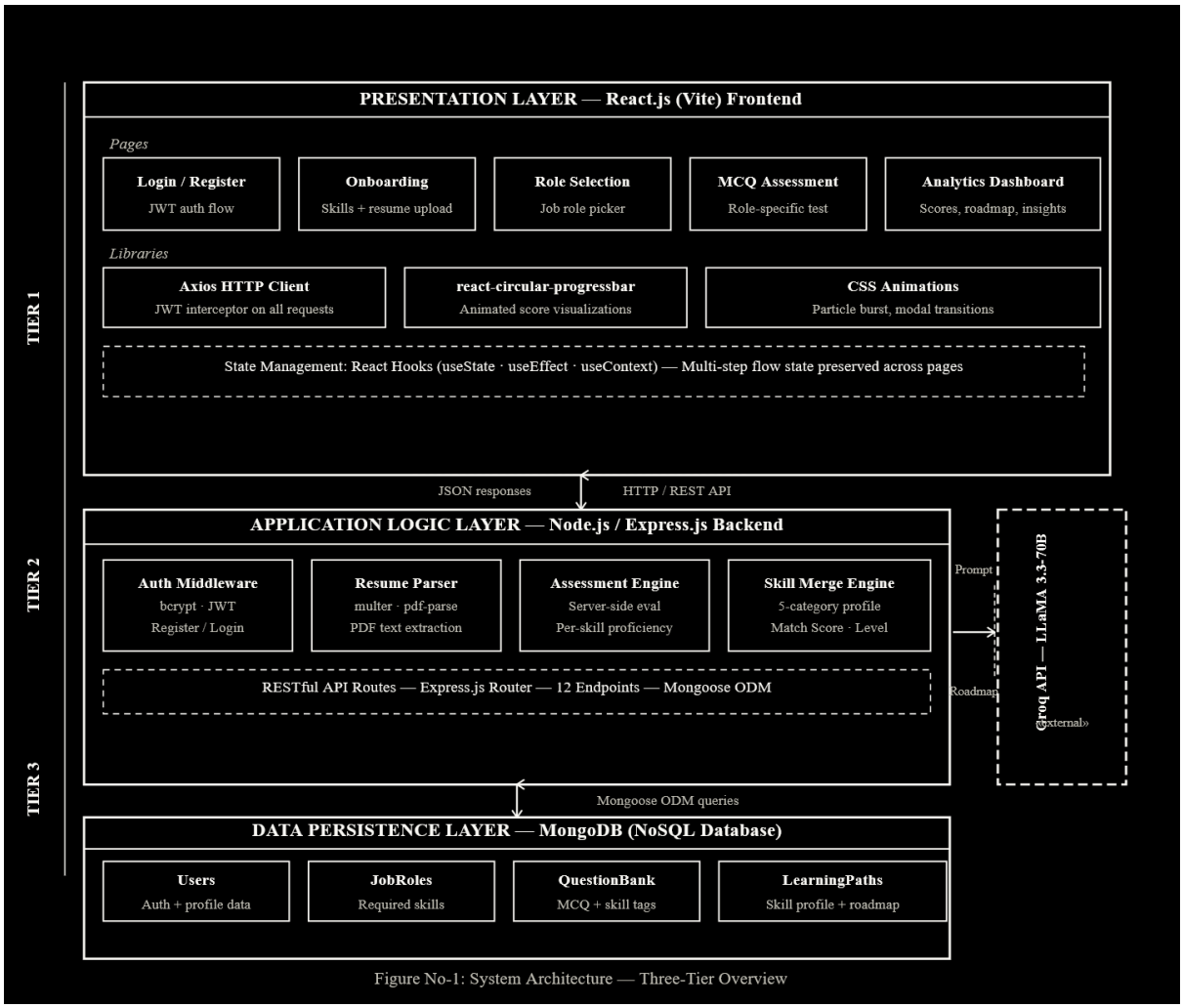


Figure No-1: System Architecture — Three-Tier Overview

Tier 1 — Presentation Layer (React.js Frontend): The frontend is built with React.js using the Vite build tool. It implements the multi-step user flow across five distinct views:

Login/Register, Onboarding, Role Selection, MCQ Assessment, and the Analytics Dashboard. State management is handled using React hooks (useState, useEffect, useContext). Axios is configured with a base URL and JWT interceptor for all API communication. UI components include animated circular progress bars, skill status badge cards, a roadmap timeline with task checkboxes, and the projected score modal.

Tier 2 — Application Logic Layer (Node.js/Express.js Backend): The backend exposes a RESTful API through Express.js with modular routing. Core modules include: authentication middleware (JWT verification), file upload handling (multer), PDF text extraction (pdf-parse), MCQ delivery and server-side evaluation, the Intelligent Skill Merging Engine (a dedicated service module), match score computation, career level determination, and the Groq AI roadmap generation service. A fallback roadmap module activates if the Groq API times out or returns an error.

Tier 3 — Data Persistence Layer (MongoDB): MongoDB stores data across four collections: Users, JobRoles, QuestionBank, and LearningPaths. Mongoose ODM manages schema definition, field validation, and query operations. The document-oriented structure of MongoDB accommodates the variable-length arrays (skills, roadmap tasks) inherent in this domain.

2 Data Flow Diagrams

2.1 DFD Level 0 — Context Diagram

The Level 0 DFD represents the entire system as a single process with three external entities:

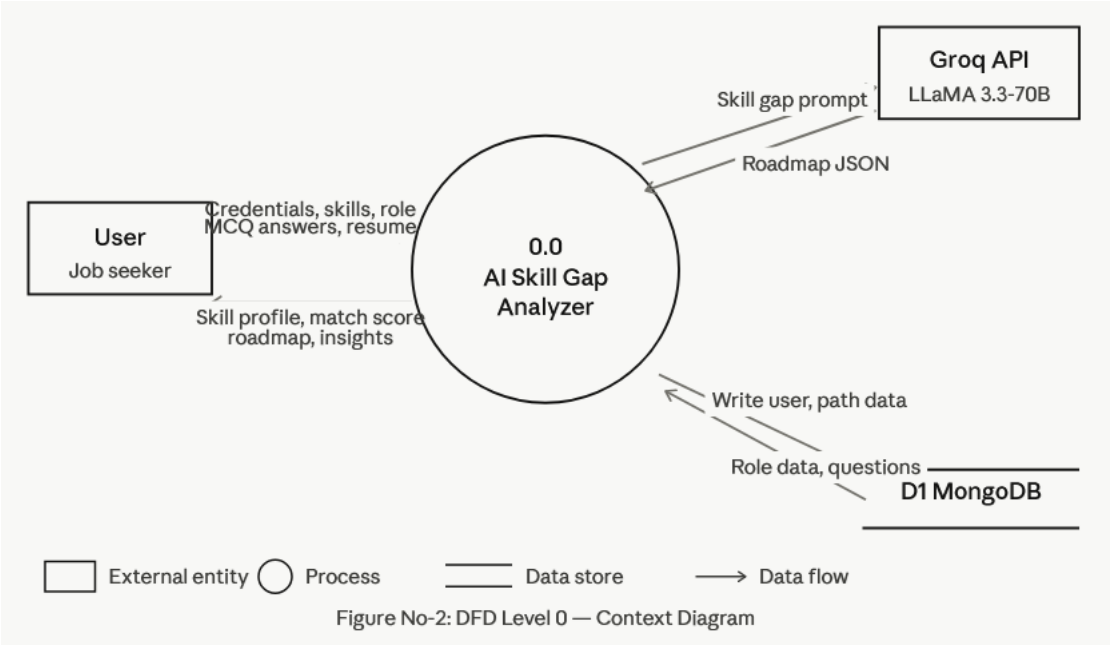


Figure No-2: DFD Level 0 — Context Diagram

External Entity 1 — User: Provides inputs including authentication credentials, personal details, declared current skills, PDF resume (optional), target job role selection, and MCQ assessment answers. Receives outputs including the Final Skill Profile, Match Score, Test Score, Career Level, AI-generated Learning Roadmap, Skill Insights, Certification Recommendations, and Projected Score.

External Entity 2 — Groq API (LLaMA 3.3-70B): Receives a structured prompt containing the user's Final Skill Profile and role context from the system. Returns a structured learning roadmap prioritizing Weak skills before Missing skills.

External Entity 3 — MongoDB Database: Provides persistent storage and retrieval of all application data across the four collections.

2.2 DFD Level 1 — Detailed Data Flow

The Level 1 DFD decomposes the main system process into eight primary sub-processes:

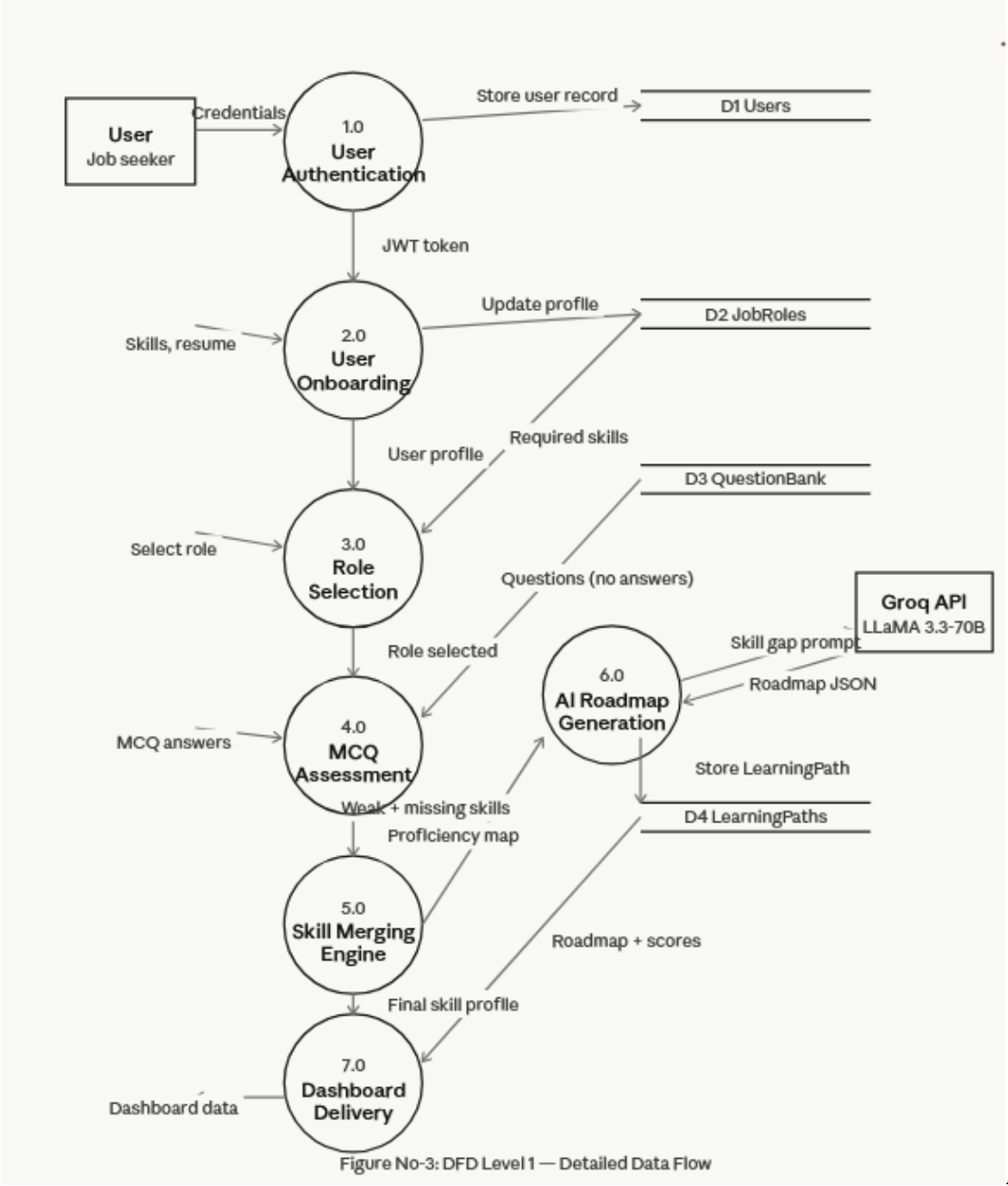


Figure No-3: DFD Level 1 — Detailed Data Flow

1. Process 1.0 — User Authentication: Accepts email/password. On registration, hashes password via bcrypt and stores to Users data store. On login, validates credentials and issues JWT. Output: JWT token to frontend.

2. Process 2.0 — User Onboarding: Collects name, email, declared skills, and optional PDF upload. pdf-parse extracts resume text. Data written to Users data store. Output: Updated user profile.

3. Process 3.0 — Role Selection: Reads all available JobRole records from JobRoles data store. Presents to user. Stores selected role to user session. Output: Selected role's required skills list.

4. Process 4.0 — MCQ Assessment: Reads role-specific questions from QuestionBank (without answer field). Delivers to user. On submission, evaluates answers server-side. Computes per-skill proficiency. Output: Skill proficiency map keyed by skillTag.

5. Process 5.0 — Skill Merging Engine: Reads declared skills and proficiency map. Reads required skills from JobRoles. Applies classification rules (see Table No. 3). Computes Match Score and Career Level. Output: Final Skill Profile object.

6. Process 6.0 — AI Roadmap Generation: Constructs structured prompt from Weak and Missing skills. Calls Groq API. Parses response into task array. Activates fallback if error. Writes LearningPath to LearningPaths data store. Output: Structured roadmap array.

7. Process 7.0 — Dashboard Delivery: Reads LearningPath record. Compiles all analysis data. Returns complete response to frontend. Output: Full dashboard data object.

8. Process 8.0 — Projected Score Simulation: Reads completed task IDs. Recalculates match score based on task completion impact. Returns projected score. Output: Current score, projected score, and gain value for modal display.

3 Entity-Relationship Diagram

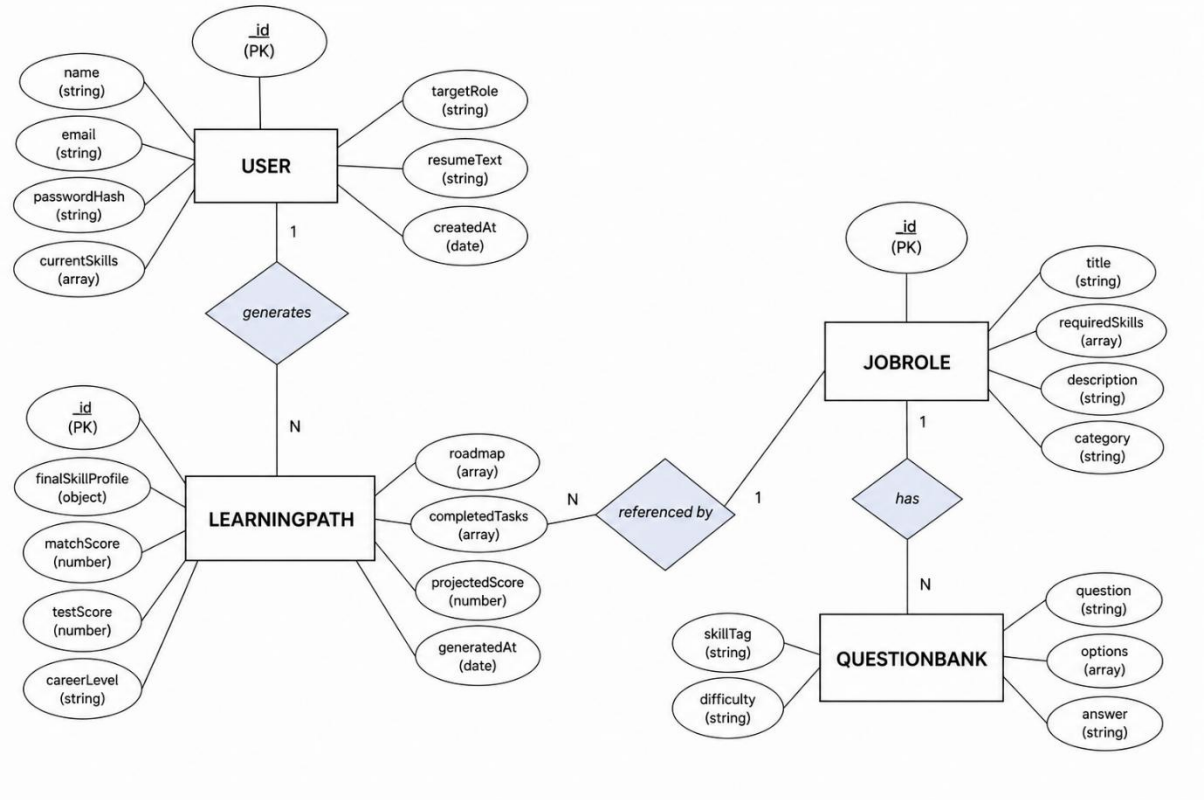


Figure No-4: Entity-Relationship Diagram (ERD)

The database design consists of four entities with the following attributes and relationships:

3.1 Entity: User

- _id (ObjectId, Primary Key)
- name (String, required)
- email (String, required, unique, indexed)
- passwordHash (String, required)

- currentSkills (Array of Strings)
- targetRole (String)
- resumeText (String, optional)
- createdAt (Date, default: now)

3.2 Entity: JobRole

- _id (ObjectId, Primary Key)
- title (String, required, unique) — e.g., Full Stack Developer
- requiredSkills (Array of Strings)
- description (String)
- category (String) — e.g., Web Development, Data Science

3.3 Entity: QuestionBank

- _id (ObjectId, Primary Key)
- role (String, required) — matches JobRole title
- question (String, required)
- options (Array of Strings, exactly 4 items)
- answer (String, required, server-side only — never sent to client)
- skillTag (String, required) — specific skill assessed, e.g., React, CSS
- difficulty (String) — Easy / Medium / Hard

3.4 Entity: LearningPath

- _id (ObjectId, Primary Key)
- userId (ObjectId, Foreign Key -> User._id)
- targetRole (String)

- finalSkillProfile (Object — keys: verified, discovered, declared, weak, missing — each an Array of Strings)
- matchScore (Number, 0-100)
- testScore (Number, 0-100)
- careerLevel (String — Beginner/Intermediate/Advanced/Expert)
- roadmap (Array of Objects — each: {skill, description, difficulty, timeEstimate})
- completedTasks (Array of Numbers — indices of completed roadmap tasks)
- projectedScore (Number, calculated on completion)
- generatedAt (Date, default: now)

Relationships:

- User to LearningPath: One-to-Many. One user can have multiple learning path records (one per analysis session). Foreign key: LearningPath.userId -> User._id.
- JobRole to LearningPath: One-to-Many. One job role can be referenced by many learning paths. Linked via targetRole string field.
- JobRole to QuestionBank: One-to-Many. One job role has many questions. Linked via role string field.

4 Use Case Diagram

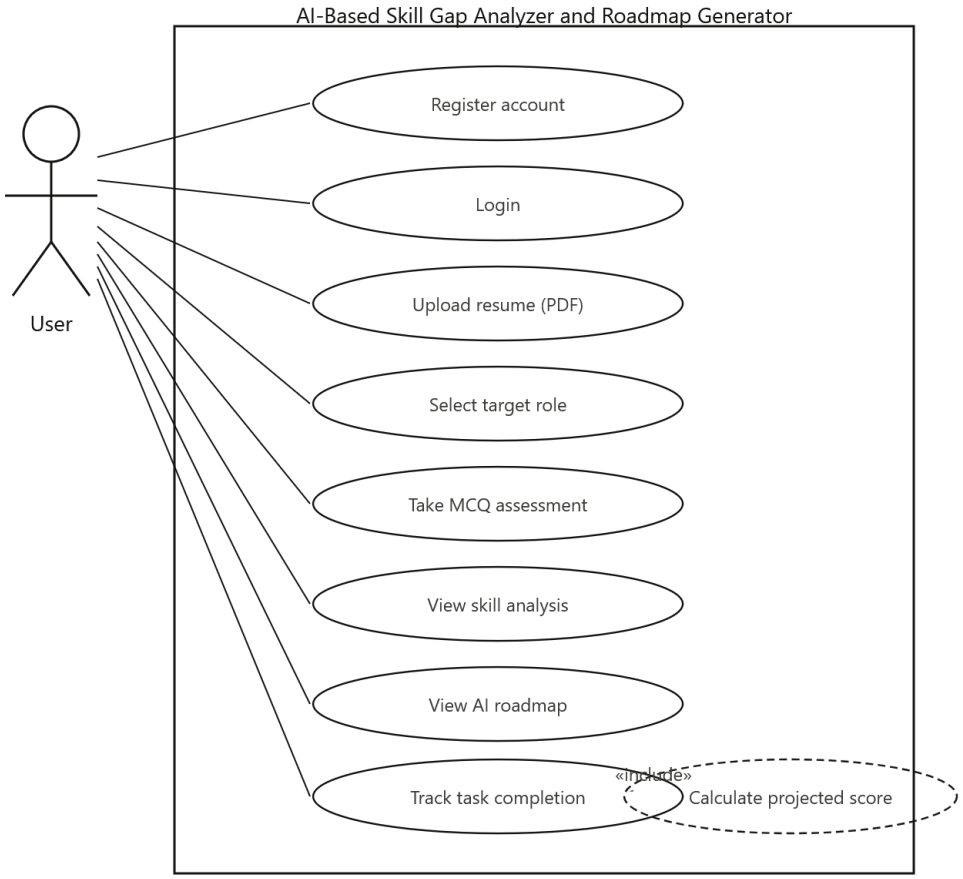


Figure No-5: Use Case Diagram

Figure No-5: Use Case Diagram

The primary actor is the User. The following use cases are identified:

- Register Account — User provides name, email, password. System validates, hashes password, stores user record, returns JWT.
- Login — User provides email, password. System validates credentials, returns JWT.
- Upload Resume — User uploads PDF. System extracts and stores text.

- Select Target Role — User selects from available job roles. System loads required skills.
- Take MCQ Assessment — User answers role-specific questions. System evaluates and scores.
- View Skill Analysis — User views Final Skill Profile, Match Score, Career Level on dashboard.
- View AI Roadmap — User views prioritized learning roadmap with difficulty and time estimates.

5 Multi-Step User Flow

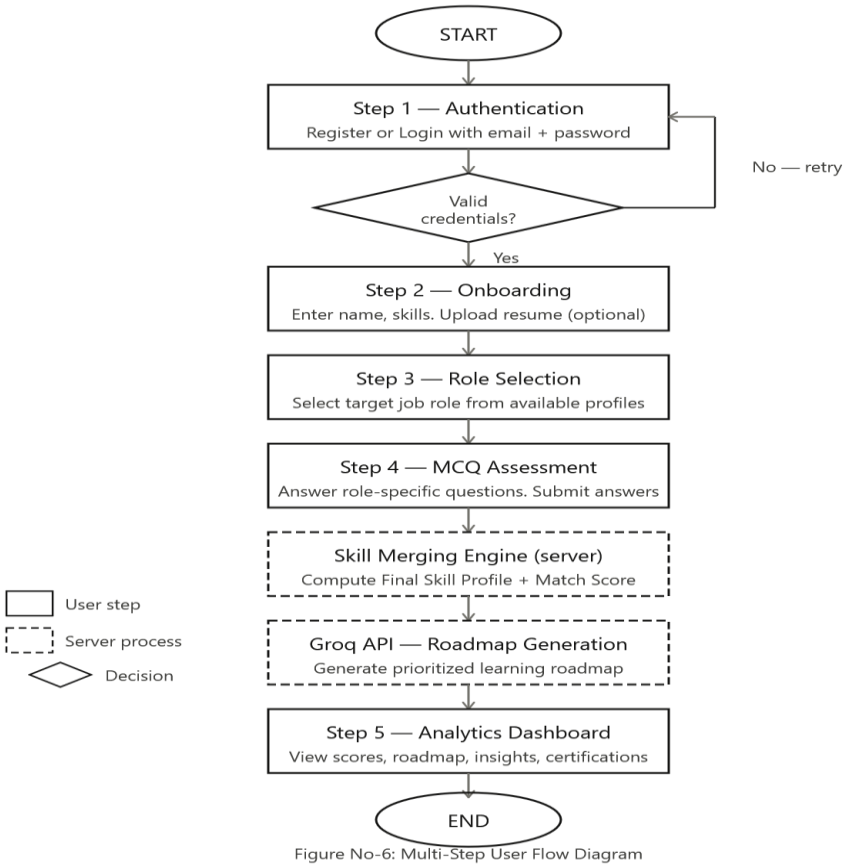


Figure No-6: Multi-Step User Flow Diagram

Figure No-6: Multi-Step User Flow Diagram

Step 1 — Authentication: User accesses the application and is presented with Login/Register options. On successful authentication, JWT is stored in browser memory and user is redirected to onboarding.

Step 2 — Onboarding: User enters name, email, and current skills. PDF resume upload is optional. System extracts text if file provided. Data is stored to the User record.

Step 3 — Role Selection: User selects a target job role from a card-based selection UI. System fetches the role's required skill set from MongoDB.

Step 4 — MCQ Skill Assessment: User is presented with role-specific MCQ questions (answers hidden). User completes assessment and submits. System evaluates responses server-side and calculates per-skill proficiency scores.

Step 5 — AI Analysis Dashboard: System runs the Skill Merging Engine, computes Match Score, calls Groq API for roadmap, and renders the complete analytics dashboard with all eleven sections.

6 Database and File Design

6.1 User Collection

Field Name	Type & Size	Validation	Description
_id	ObjectId	Auto	Primary key — auto-generated by MongoDB
name	String, max 100	Required	Full name of the user
email	String, max 150	Required, Unique	User email address — used for login
passwordHash	String, 60	Required	bcrypt hash of user password (10 salt rounds)
currentSkills	Array[String]	Optional	User-declared current skills
targetRole	String, max 100	Optional	Selected target job role
resumeText	String, max 10000	Optional	Plain text extracted from uploaded PDF resume

createdAt	Date	Auto	Timestamp of account creation

Table No. 4: User Collection — Database Field Design

6.2 JobRole Collection

Field Name	Type & Size	Validation	Description
_id	ObjectId	Auto	Primary key
title	String, max 100	Required, Unique	Job role name e.g., Full Stack Developer
requiredSkills	Array[String]	Required	List of skills required for this role
description	String, max 500	Optional	Brief role description
category	String, max 50	Optional	Domain category e.g., Web Development

Table No. 5: JobRole Collection — Database Field Design

6.3 QuestionBank Collection

Field Name	Type & Size	Validation	Description
_id	ObjectId	Auto	Primary key
role	String, max 100	Required	Target job role this question belongs to
question	String, max 500	Required	Question text
options	Array[String], 4	Required	Four answer options (A, B, C, D)
answer	String, max 200	Required	Correct answer — stored server-side only, never sent to client
skillTag	String, max 50	Required	Specific skill this question assesses e.g., React, CSS
difficulty	String, max 10	Optional	Easy / Medium / Hard

Table No. 6: QuestionBank Collection — Database Field Design

6.4 LearningPath Collection

Field Name	Type & Size	Validation	Description
_id	ObjectId	Auto	Primary key
userId	ObjectId (ref: User)	Required	Foreign key linking to User collection
targetRole	String, max 100	Required	The job role analyzed
finalSkillProfile	Object	Required	Five-category skill classification object
matchScore	Number (0-100)	Required	Skill coverage percentage
testScore	Number (0-100)	Required	Overall assessment score percentage
careerLevel	String, max 20	Required	Beginner / Intermediate / Advanced / Expert
roadmap	Array[Object]	Required	AI-generated task list with skill, description, difficulty, timeEstimate
completedTasks	Array[Number]	Optional	Indices of completed roadmap tasks
projectedScore	Number (0-100)	Optional	Calculated projected match score after task completion
generatedAt	Date	Auto	Timestamp of roadmap generation

Table No. 7: LearningPath Collection — Database Field Design

7 Intelligent Skill Merging Engine — Processing Logic

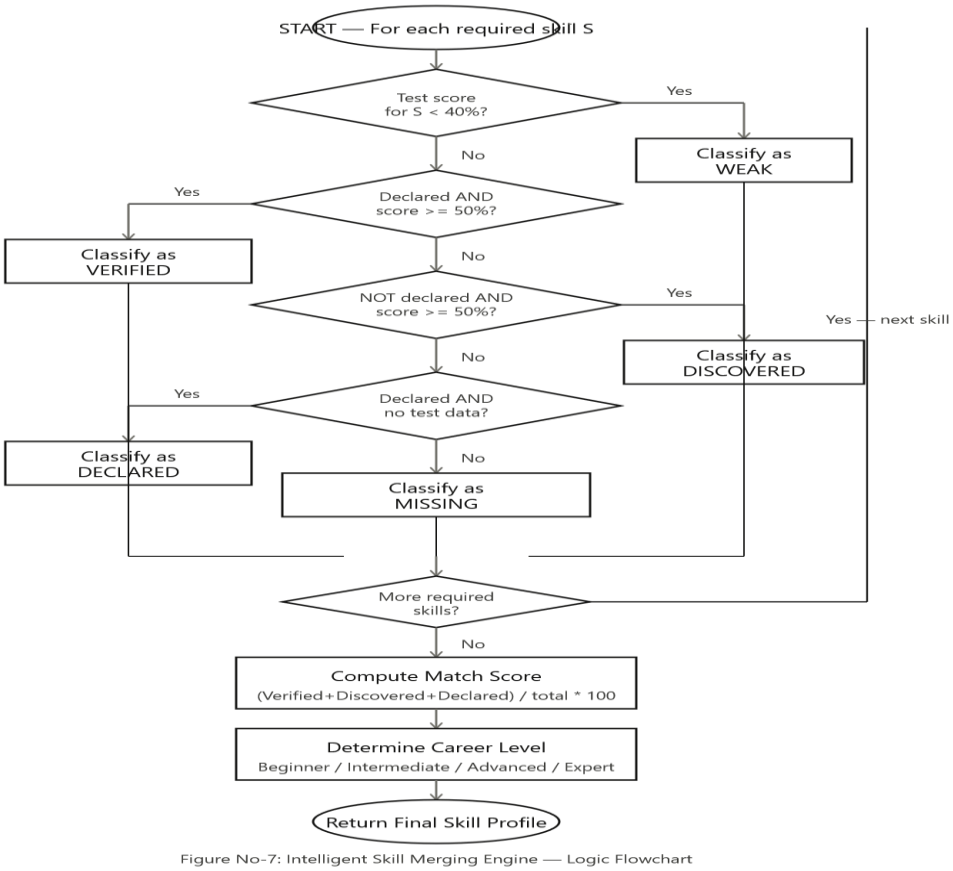


Figure No-7: Intelligent Skill Merging Engine — Logic Flowchart

The Intelligent Skill Merging Engine follows this processing logic for each required skill of the target role:

- For each skill S in JobRole.requiredSkills:
- Check if user declared S (declared = currentSkills.includes(S)).
- Check if S exists in test proficiency map (tested = proficiencyMap[S] !== undefined).
- If tested AND proficiencyMap[S] < 40%: classify as WEAK regardless of declaration.
- Else if declared AND tested AND proficiencyMap[S] >= 50%: classify as VERIFIED.
- Else if NOT declared AND tested AND proficiencyMap[S] >= 50%: classify as DISCOVERED.
- Else if declared AND NOT tested: classify as DECLARED.

- Else: classify as MISSING.
- After all skills classified: Match Score = (VERIFIED + DISCOVERED + DECLARED).length / requiredSkills.length * 100.
- Career Level: Score >= 80% = Expert; >= 60% = Advanced; >= 40% = Intermediate; else = Beginner.

CHAPTER-4

SYSTEM DEVELOPMENT

1 Programming Language and Platform

The system is developed using JavaScript as the primary programming language across both frontend and backend, enabling code reusability and a unified development experience. The frontend employs React.js with Vite as the build tool, and the backend uses Node.js with Express.js as the application framework. MongoDB serves as the NoSQL database with Mongoose as the ODM layer. This combination — commonly referred to as the MERN stack — is a mature, widely adopted technology stack for full-stack web application development.

2 Development Methodology — Agile Sprints

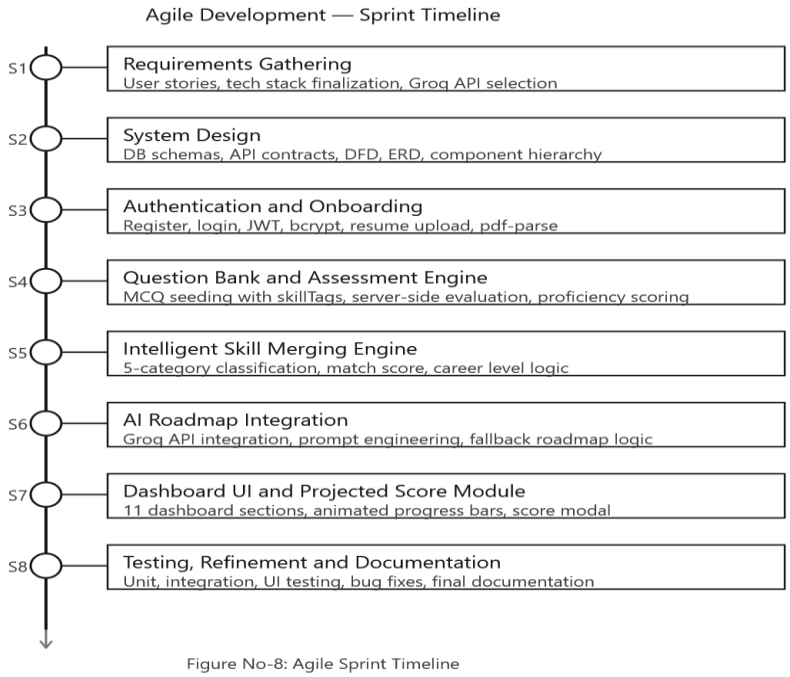


Figure No-8: Agile Sprint Timeline

2.1 Sprint 1 — Requirements Gathering and Tech Stack Finalization

User stories were identified and prioritized. The Groq API (LLaMA 3.3-70B) was selected over alternatives based on free-tier availability, inference speed, and structured output quality. MongoDB was chosen over SQL databases due to the variable-length array fields (skills, roadmap tasks) in the data model. Project scaffolding was initialized using create-react-app (Vite template) for the frontend and npm init for the backend.

2.2 Sprint 2 — System Design

All four MongoDB schemas were designed and implemented using Mongoose. REST API endpoints were defined with request/response specifications. The React component hierarchy was mapped: App -> AuthGuard -> MultiStepFlow -> [LoginPage, OnboardingPage, RoleSelectPage, AssessmentPage, DashboardPage]. State management strategy using React Context and useState hooks was finalized.

2.3 Sprint 3 — Authentication and Onboarding

Backend: POST /api/auth/register — accepts name, email, password; hashes password with bcrypt (10 salt rounds); stores user; returns JWT. POST /api/auth/login — validates credentials; returns JWT. Auth middleware function extracts and verifies JWT from Authorization header for protected routes.

Frontend: Login and Register forms with client-side validation. JWT stored in React Context (not localStorage per security considerations). Route guard redirects unauthenticated users to login. Onboarding form with skill tag input and PDF file upload. Axios POST to /api/users/onboard with multipart/form-data encoding.

2.4 Sprint 4 — Question Bank and Assessment Engine

Backend: Question bank seeded for common roles (Full Stack Developer, Data Scientist, DevOps Engineer) with 15-20 questions per role, each tagged with a skillTag. GET

/api/assessment/:role — fetches questions using MongoDB projection to exclude the answer field. POST /api/assessment/submit — receives answer array; evaluates against stored answers; groups by skillTag; computes per-skill proficiency.

Proficiency formula: For each skillTag, proficiency = (correctCount / totalCount) * 100. Classification: >= 70% = Strong, 40-69% = Moderate, < 40% = Weak.

2.5 Sprint 5 — Intelligent Skill Merging Engine

The merging engine was implemented as a standalone service module (skillMerger.js) with a single exported function: mergeSkills(declaredSkills, proficiencyMap, requiredSkills). The function returns a finalSkillProfile object with five arrays (verified, discovered, declared, weak, missing) and computed matchScore and careerLevel values. The service is stateless and purely functional, facilitating unit testing without database dependencies.

2.6 Sprint 6 — AI Roadmap Generation

The Groq API was integrated using the groq-sdk npm package. The prompt construction function generates a structured prompt specifying: the target role, the list of Weak skills (with their proficiency scores), the list of Missing skills, and instructions for the output format (JSON array of task objects with skill, description, difficulty, timeEstimate fields).

The response parser extracts the JSON array from the LLM response text. A fallback roadmap generator produces a rule-based roadmap (one task per Weak/Missing skill with predefined descriptions and time estimates) if the Groq API call fails or times out after 10 seconds.

2.7 Sprint 7 — Dashboard UI and Projected Score Module

The React dashboard renders eleven sections using modular components. The react-circular-progressbar library renders animated Match Score and Projected Score circles. Skill status badges use color-coded styling (green = Verified, blue = Discovered, grey = Declared, orange = Weak, red = Missing). The roadmap timeline renders each task as a card with skill name,

description, difficulty badge, time estimate, and a checkbox. The Projected Score module tracks completedTasks in state; when all tasks are checked, the Calculate Projected Score button activates and triggers POST /api/roadmap/projected-score.

2.8 Sprint 8 — Testing and Refinement

Comprehensive testing was conducted across unit, integration, and UI levels. Bugs identified during testing were documented, prioritized, and resolved. Edge cases including empty skill arrays, all-correct and all-incorrect assessment submissions, API timeout simulation, and invalid JWT handling were all addressed.

3 REST API Endpoint Reference

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register new user account
POST	/api/auth/login	No	Login and receive JWT token
POST	/api/users/onboard	Yes	Submit onboarding data with optional resume PDF
GET	/api/roles	Yes	Fetch all available job role profiles
GET	/api/assessment/:role	Yes	Fetch role-specific MCQ questions (no answers)
POST	/api/assessment/submit	Yes	Submit answers; receive per-skill proficiency scores
POST	/api/analyze	Yes	Run skill merging engine; generate AI roadmap; return dashboard data
GET	/api/dashboard/:userId	Yes	Retrieve stored learning path for dashboard rendering
PATCH	/api/roadmap/tasks	Yes	Update completed task indices in LearningPath
POST	/api/roadmap/projected-score	Yes	Calculate projected score based on completed tasks
DELETE	/api/users/:id	Yes	Delete user account and associated data

Table No. 8: REST API Endpoint Reference

4 Tools and Technologies

Technology / Tool	Category	Purpose
React.js (Vite)	Frontend Framework	Multi-step UI, component-based architecture, HMR during development
Axios	HTTP Client	API communication with JWT interceptor from frontend to backend
react-circular-progressbar	UI Component	Animated circular progress bars for Match Score and Projected Score display
Node.js	Runtime	Server-side JavaScript execution environment
Express.js	Backend Framework	RESTful API routing and middleware management
MongoDB	NoSQL Database	Persistent storage for all four collections
Mongoose	ODM	Schema definition, validation, and query abstraction
Groq API (LLaMA 3.3-70B)	AI / LLM Service	Generating prioritized, structured learning roadmaps
multer	File Upload	Handling PDF resume file uploads on backend
pdf-parse	PDF Processing	Extracting plain text from uploaded PDF resumes
bcrypt	Security	Password hashing with 10 salt rounds for secure storage
JSON Web Tokens (JWT)	Authentication	Stateless session management and protected route authorization
Postman	API Testing	Testing and validating all REST API endpoints
Git / GitHub	Version Control	Source code management and version history
VS Code	IDE	Primary development and debugging environment

Table No. 9: Tools and Technologies Used

5 Query Language

5.1 Skill Mergining Engine :

Analysis controller.js

```
    const finalSkillProfile = {};


    requiredSkills.forEach(reqSkill => {
      const userDeclared = skillNames.some(u => skillMatches(u, reqSkill));


      // Find matching test skill (fuzzy)
           const testSkillKey = Object.keys(skillBreakdown).find(k =>
skillMatches(k, reqSkill));
      const testData = testSkillKey ? skillBreakdown[testSkillKey] : null;
      const testPct = testData ? testData.proficiency : null;


      if (testPct !== null && testPct < 40) {
        finalSkillProfile[reqSkill] = 'Weak';
      } else if (userDeclared && testPct !== null && testPct >= 50) {
        finalSkillProfile[reqSkill] = 'Verified';
      } else if (!userDeclared && testPct !== null && testPct >= 50) {
        finalSkillProfile[reqSkill] = 'Discovered';
      } else if (userDeclared) {
        finalSkillProfile[reqSkill] = 'Declared';
      } else {
        finalSkillProfile[reqSkill] = 'Missing';
      }
    });
```

```
        level: testData ? Math.round(testData.proficiency / 20) : (status
=== 'Declared' ? 3 : 0),
        status
          };       const  discoveredSkills  =  requiredSkills.filter(s  =>
finalSkillProfile[s] === 'Discovered');
    const verifiedSkills   = requiredSkills.filter(s => finalSkillProfile[s]
=== 'Verified');

    const insightLines = [];
    if (verifiedSkills.length > 0)
      insightLines.push(`Your declared skills in ${verifiedSkills.slice(0,
3).join(', ')} were confirmed by the test.`);
    if (discoveredSkills.length > 0)
           insightLines.push(`We  discovered  you  have  knowledge  of
${discoveredSkills.join(', ')} based on your test performance — even though
you didn't list ${discoveredSkills.length === 1 ? 'it' : 'them'}.`);
    if (weakSkills.length > 0)    let roadmap = [];
    let recommendedCertifications = [];

    // Prioritize: weak skills first, then missing
    const prioritizedGaps = [
      ...weakSkills.map(s => `${s} (Weak — needs improvement)`),
      ...missingSkills.map(s => `${s} (Missing)`)
    ];

    try {
```

```
      const prompt = `You are an AI career advisor. Analyze this profile and
return a JSON learning roadmap.

Target Role: ${targetRole}
Declared Skills: ${skillNames.join(', ') || 'None'}
Verified Skills (declared + test confirmed): ${verifiedSkills.join(', ') ||
'None'}
Discovered Skills (found via test): ${discoveredSkills.join(', ') || 'None'}
Weak Skills (test score < 40%): ${weakSkills.join(', ') || 'None'}
Missing Skills: ${missingSkills.join(', ') || 'None'}
Match Score: ${matchScore}%
${assessmentResult    ?    `Test    Score:    ${assessmentResult.percentage}%
(${assessmentResult.skillLevel})` : 'No test taken'}

IMPORTANT: Prioritize weak skills FIRST in the roadmap (they need fixing
before moving forward), then missing skills.

Return ONLY this JSON structure:
{
  "roadmap": [
    {
      "month": "Month 1",
Certification', 'Online Professional Course'];
    }

    const learningPath = new LearningPath({
      userId: req.userId || user._id,
      targetRole,
```

```
      matchScore,
      careerLevel,
      missingSkills: categorizedMissing,
      roadmap: { roadmap, recommendedCertifications }
    });
    await learningPath.save();
```

5.2 Assessment evaluation :

assessmentRoutes.js :

```
const express = require('express');
const router = express.Router();
const questionBank = require('../data/questionBank');

// GET questions for a role — strip answer, keep skill tag
router.get('/:role', (req, res) => {
  const role = decodeURIComponent(req.params.role);
  const questions = questionBank[role];
  if (!questions) return res.status(404).json({ error: 'No questions found for this role' });
  const sanitized = questions.map(({ question, options, skill }) => ({ question, options, skill }));
  res.json(sanitized);
});
// POST submit answers — returns score + skill-wise breakdown
router.post('/submit', (req, res) => {
  const { role, answers } = req.body;
```

```
const questions = questionBank[role];
if (!questions) return res.status(404).json({ error: 'Role not found' });
let score = 0;
const skillMap = {};   // { CSS: { correct: 2, total: 3 }, ... }
const weakAreas = [];
answers.forEach((userAnswer, i) => {
  const q = questions[i];
  if (!q) return;

  const skill = q.skill || 'General';
  if (!skillMap[skill]) skillMap[skill] = { correct: 0, total: 0 };
  skillMap[skill].total++;
  if (userAnswer === q.answer) {
    score++;
    skillMap[skill].correct++;
  } else {
    weakAreas.push(q.question);
  }
});
// Build skill breakdown with proficiency % and classification
const skillBreakdown = {};
Object.entries(skillMap).forEach(([skill, { correct, total }]) => {
  const proficiency = Math.round((correct / total) * 100);
  let level;
```

```
    if (proficiency >= 70) level = 'Strong';
    else if (proficiency >= 40) level = 'Moderate';
    else level = 'Weak';
    skillBreakdown[skill] = { correct, total, proficiency, level };
  });
  const percentage = Math.round((score / questions.length) * 100);
  let skillLevel = 'Beginner';
  if (percentage >= 80) skillLevel = 'Advanced';
  else if (percentage >= 50) skillLevel = 'Intermediate';

  res.json({ score, total: questions.length, percentage, skillLevel,
weakAreas, skillBreakdown });
});
```

5.3 Groq API integration:

groq.js :

```
const Groq = require('groq-sdk');
const groq = new Groq({
  apiKey: process.env.GROQ_API_KEY
});
module.exports = groq;
```

groqController.js :

```
    const discoveredSkills = requiredSkills.filter(s =>
finalSkillProfile[s] === 'Discovered');
```

```
    const verifiedSkills   = requiredSkills.filter(s =>
finalSkillProfile[s] === 'Verified');


    const insightLines = [];
    if (verifiedSkills.length > 0)
      insightLines.push(`Your declared skills in ${verifiedSkills.slice(0,
3).join(', ')} were confirmed by the test.`);
    if (discoveredSkills.length > 0)
      insightLines.push(`We discovered you have knowledge of
${discoveredSkills.join(', ')} based on your test performance — even though
you didn't list ${discoveredSkills.length === 1 ? 'it' : 'them'}.`);
    if (weakSkills.length > 0)
      insightLines.push(`Your understanding of ${weakSkills.slice(0,
3).join(', ')} is weak based on test results and needs improvement.`)
  "recommendedCertifications": ["cert1", "cert2", "cert3"]
}`;
      const completion = await groq.chat.completions.create({
        model: 'llama-3.3-70b-versatile',
        messages: [{ role: 'user', content: prompt }],
        temperature: 0.7,
        response_format: { type: 'json_object' }
      });
      const aiResponse = JSON.parse(completion.choices[0].message.content);
      };
```

5.4 Auth middleware:

auth.js :

```
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided' });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid token' });
  }
};
```

5.5 MongoDB schemas:

User.js :

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String },
  currentSkills: [String],
  targetRole: String,
  resumeText: String,
```

```
  createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('User', userSchema);
```

JobRole.js:

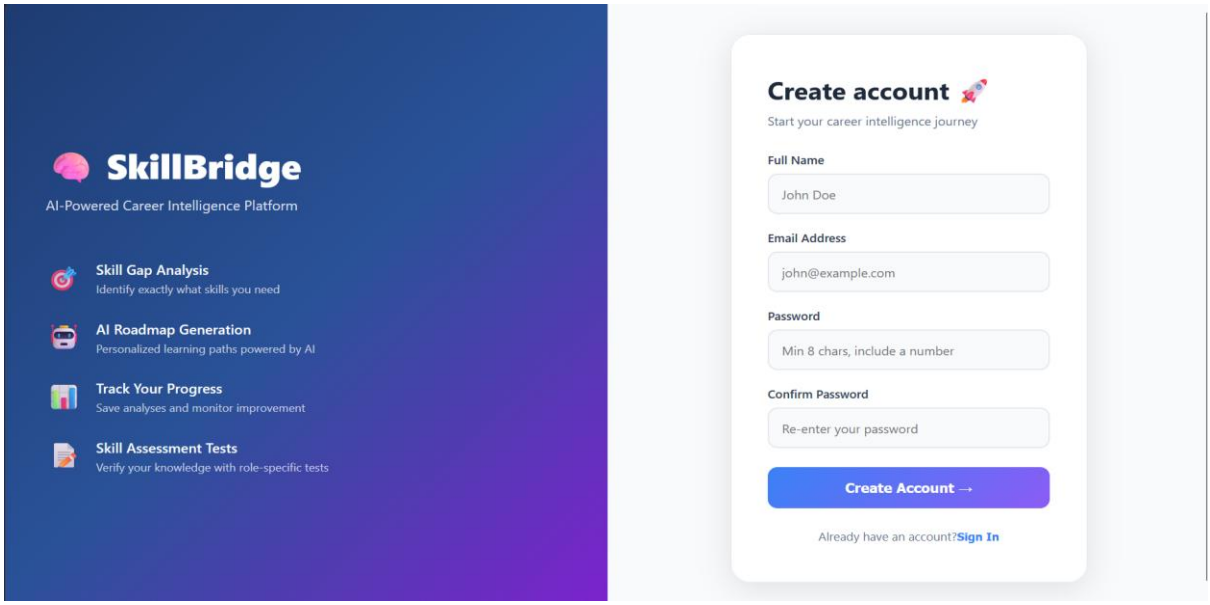
```
const mongoose = require('mongoose');
const jobRoleSchema = new mongoose.Schema({
  title: { type: String, required: true, unique: true },
  requiredSkills: [String],
  description: String
});
module.exports = mongoose.model('JobRole', jobRoleSchema);
```

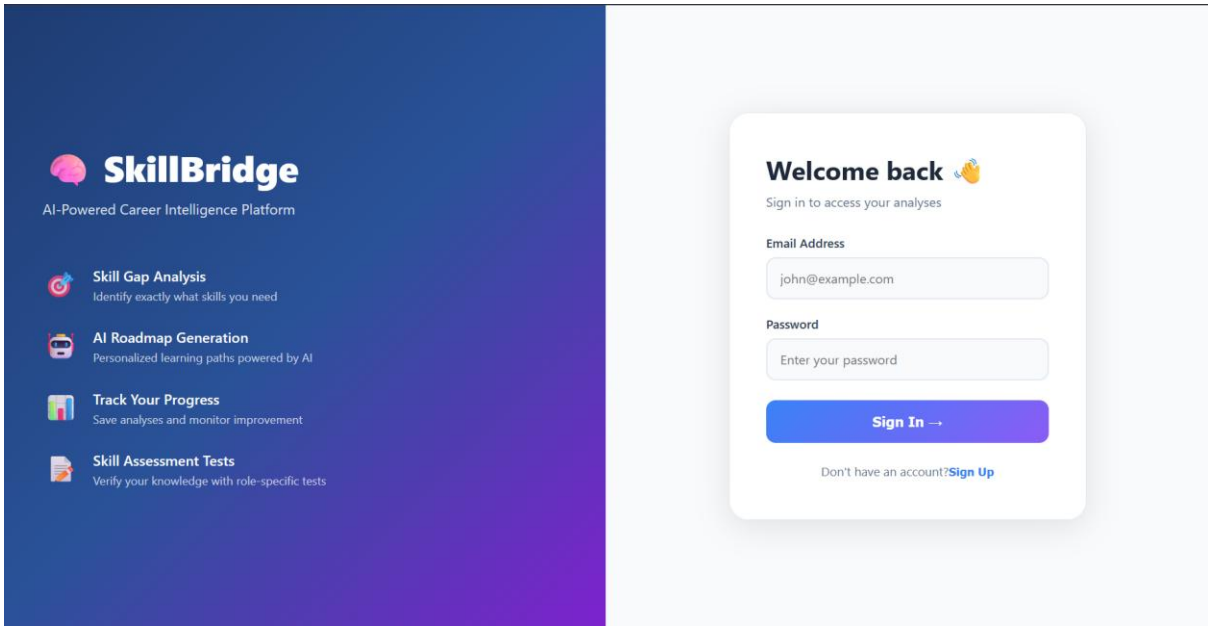
LearningPath.js:

```
const mongoose = require('mongoose');
const learningPathSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  targetRole: String,
  matchScore: Number,
  careerLevel: String,
  missingSkills: mongoose.Schema.Types.Mixed,
  roadmap: mongoose.Schema.Types.Mixed,
  createdAt: { type: Date, default: Date.now }
});
```

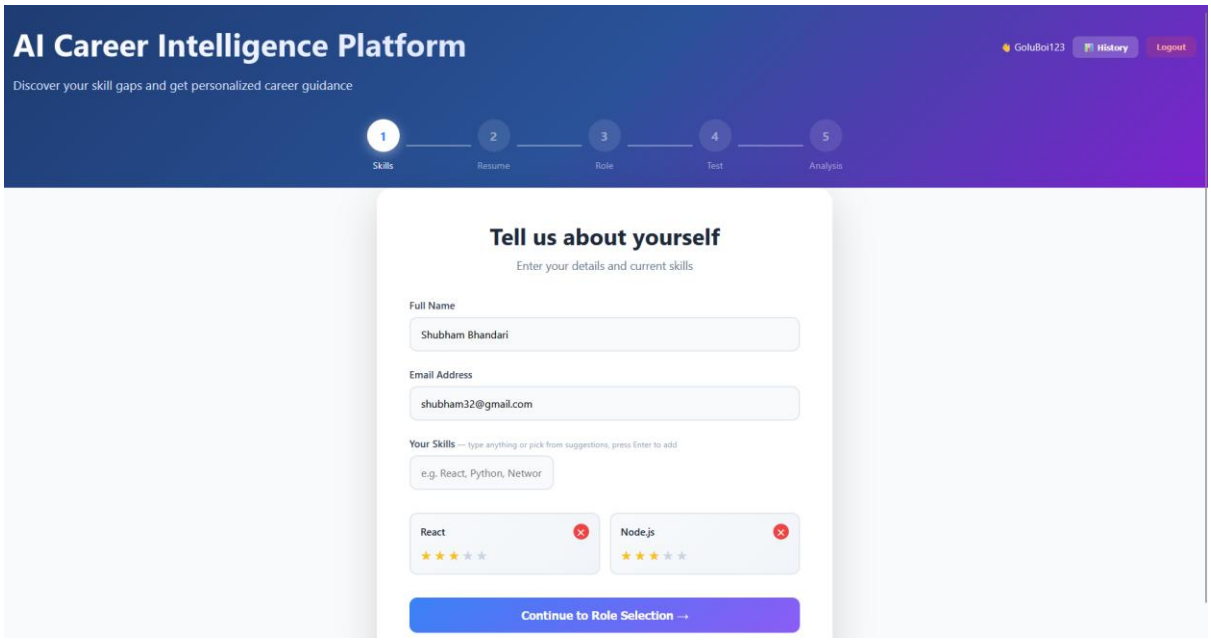
6. OUTPUT

Login and Registration Page

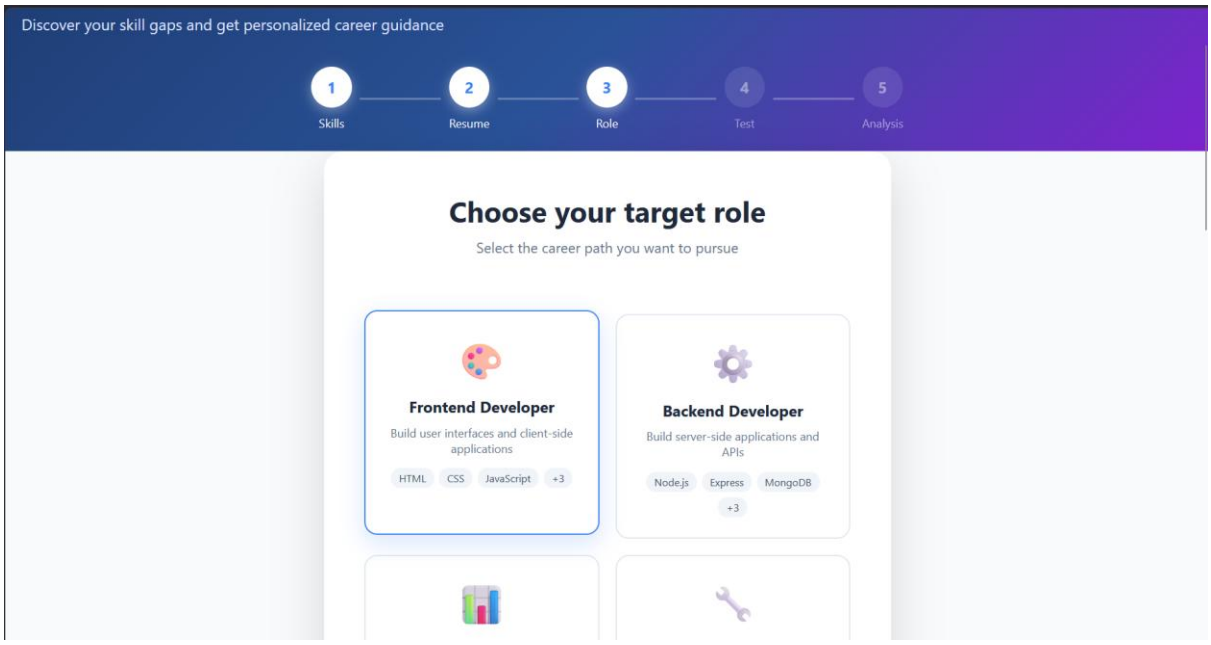




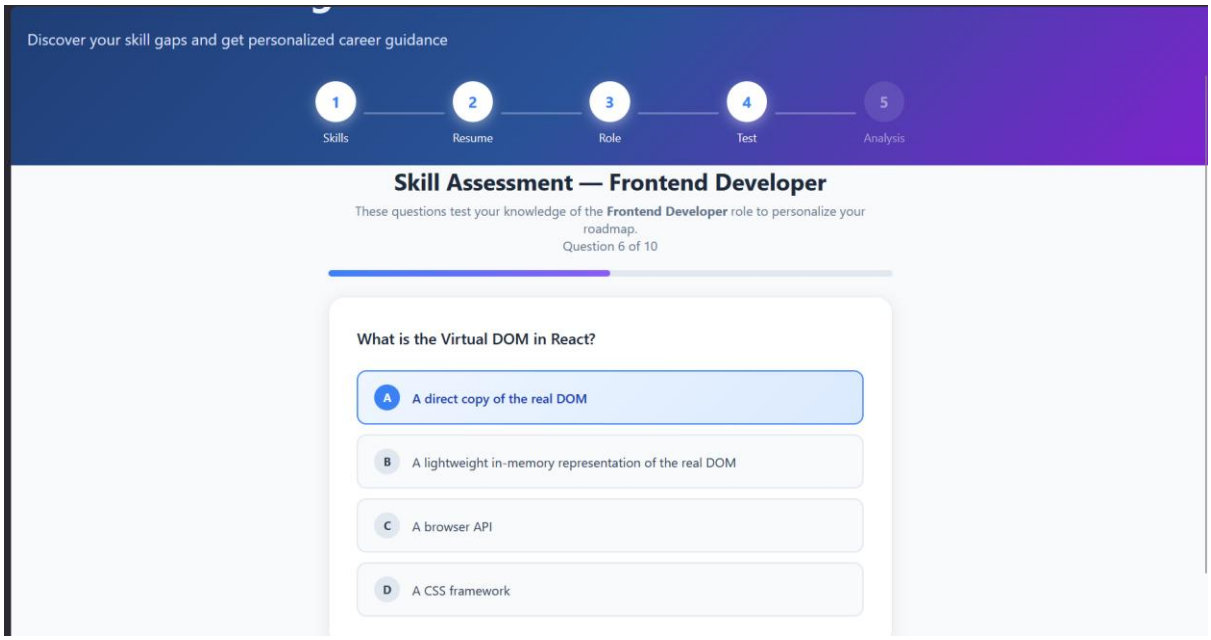
User Onboarding Form

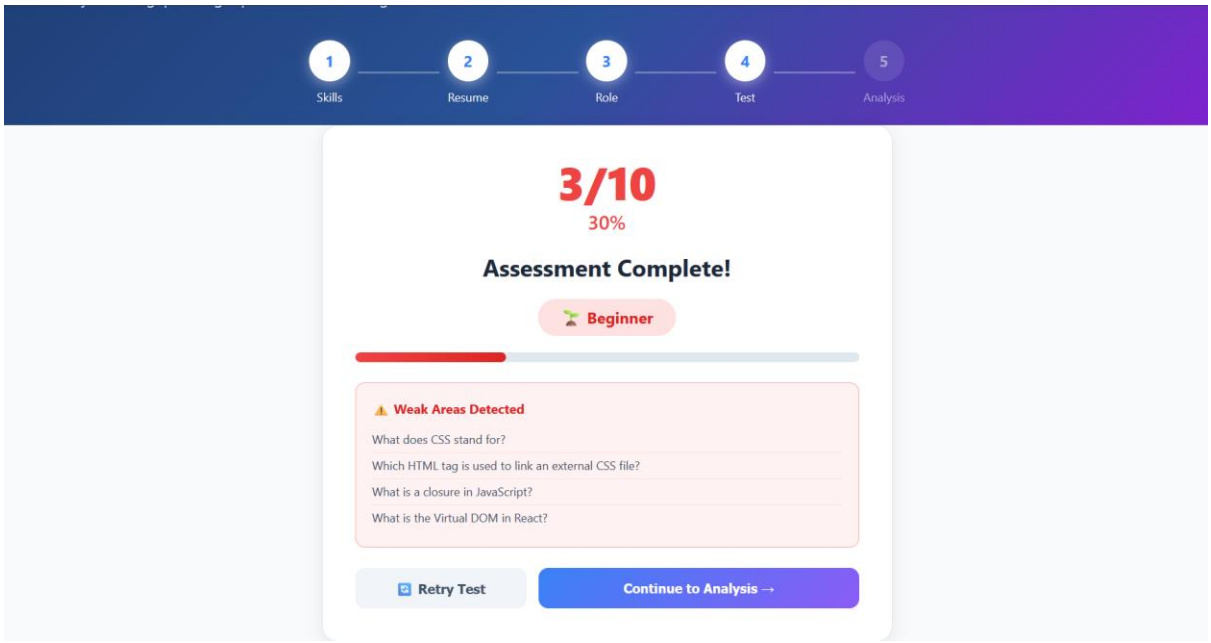


Role Selection Page

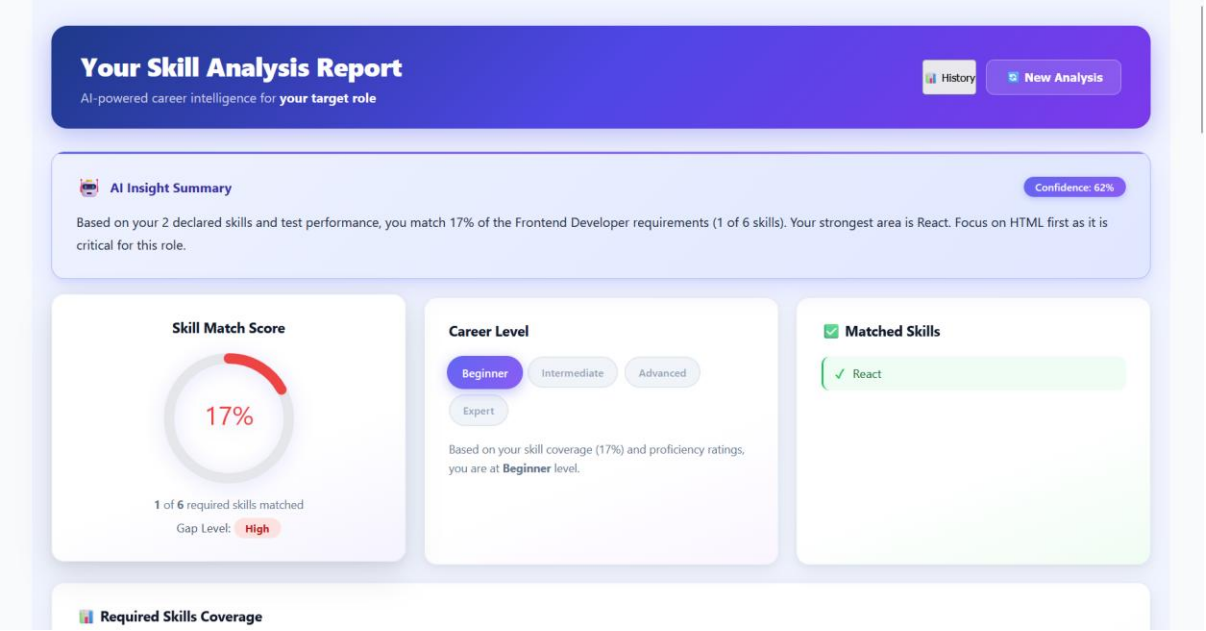


MCQ Assessment Screen

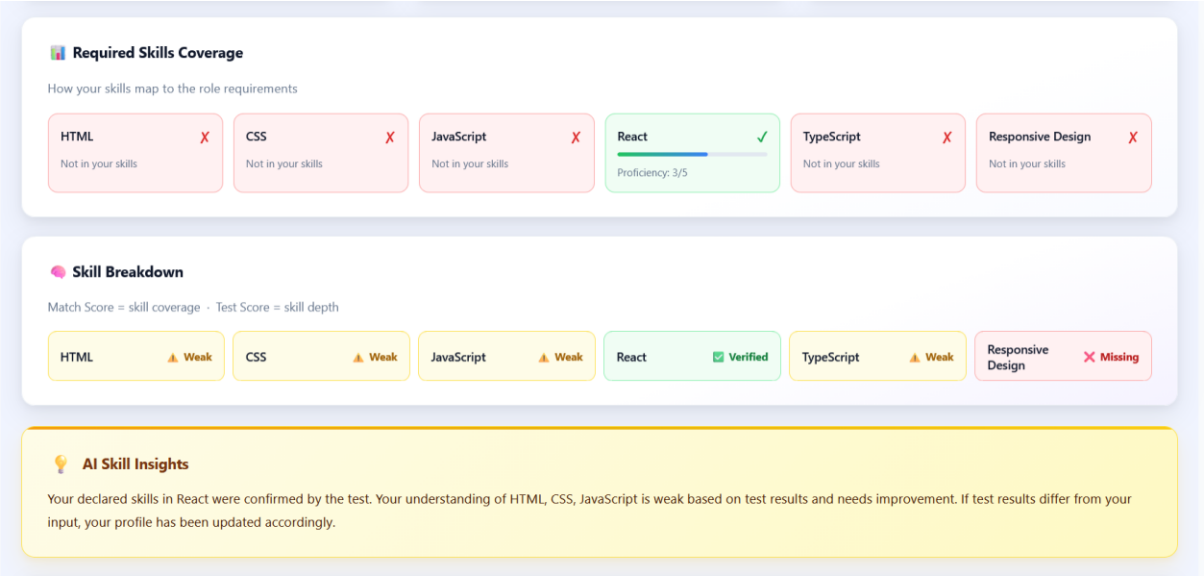




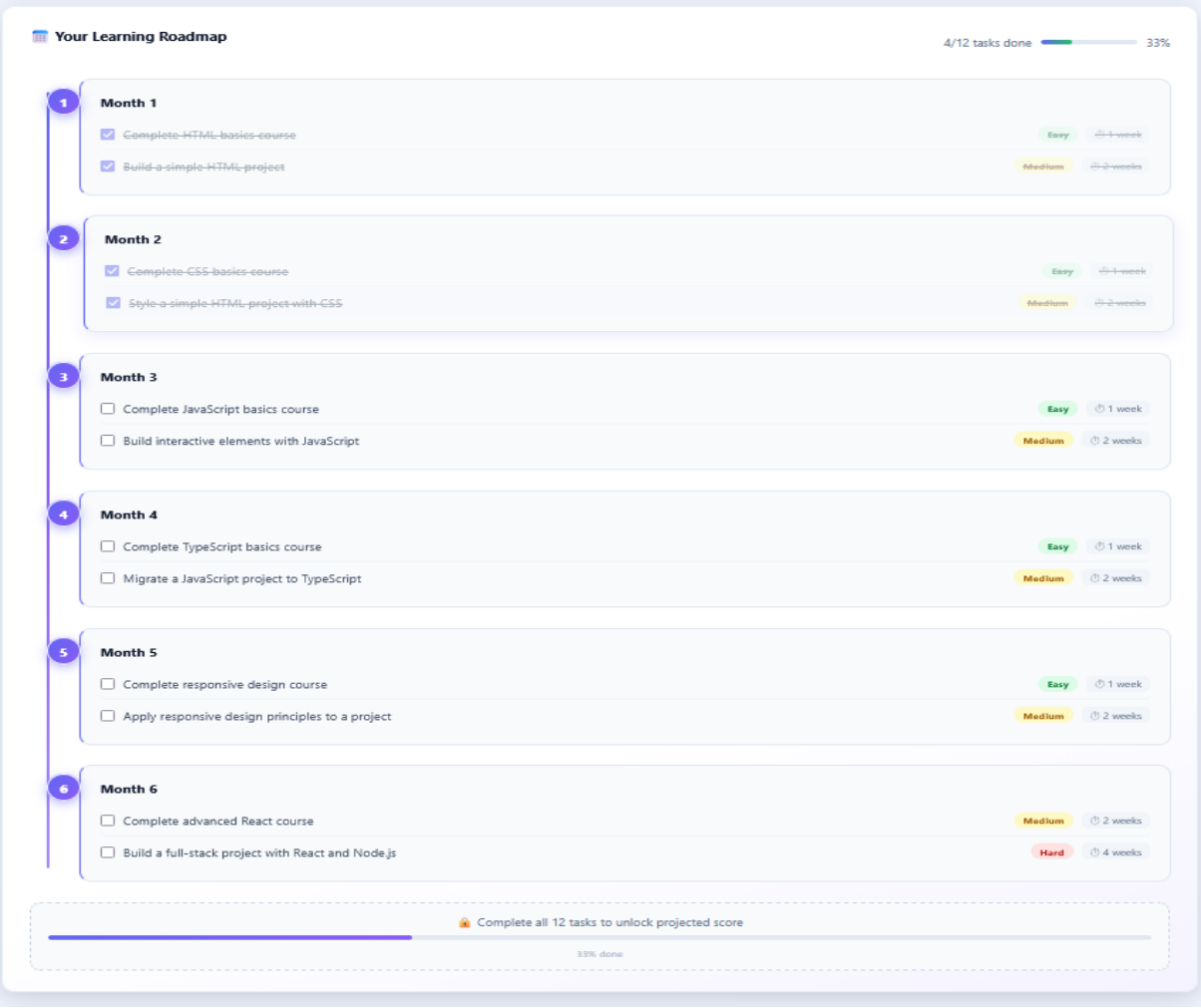
Analytics Dashboard — Score and Career Level



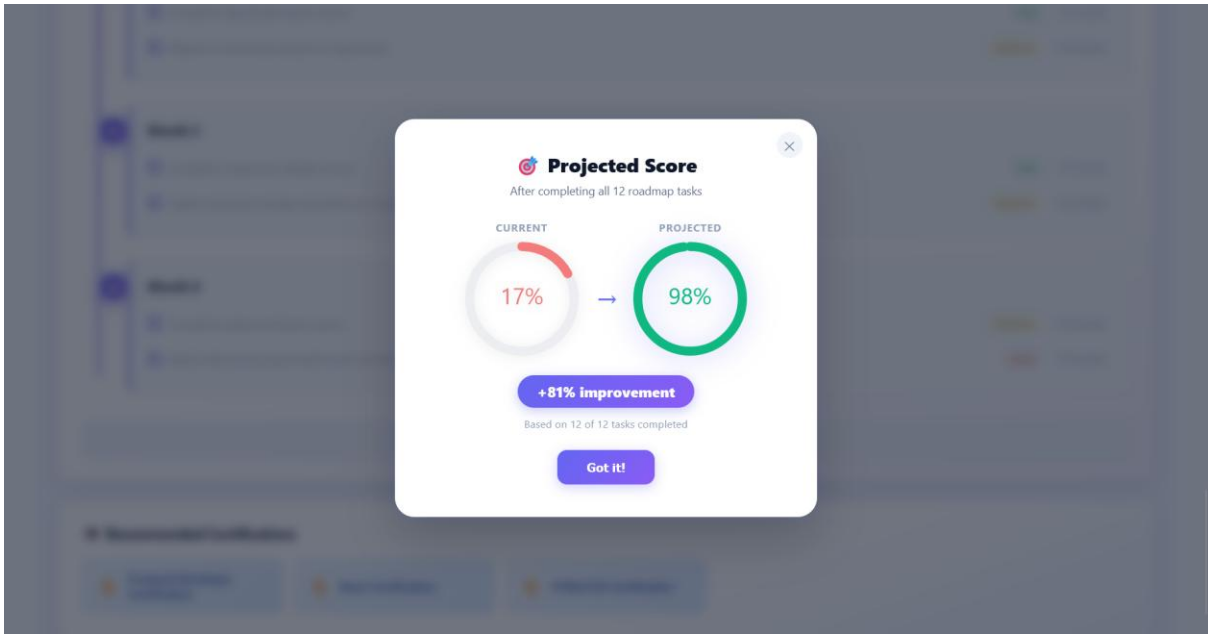
Skill Breakdown Section



Learning Roadmap Section



Projected Score Modal



CHAPTER-5

SYSTEM TESTING

System testing is a critical phase in the software development lifecycle where the complete and integrated application is tested to evaluate whether it meets the specified requirements. For the AI-Based Skill Gap Analyzer and Roadmap Generator, system testing was performed to verify that all core modules — user authentication, skill assessment, intelligent skill merging, AI roadmap generation, and the analytics dashboard — function correctly and produce accurate, consistent results. Testing was carried out manually across all modules including registration and login, MCQ assessment delivery and evaluation, skill merging logic, Groq API integration, projected score simulation, and task tracking. Every feature was tested against both valid and invalid inputs to ensure the system handles all scenarios gracefully. The goal was to identify and resolve any functional errors, security vulnerabilities, or logic inconsistencies before the final submission of the application.

6.1 TESTING STRATEGY

The testing strategy for the AI-Based Skill Gap Analyzer was based on a manual black-box testing approach where the system was tested from the end-user's perspective without examining the internal code structure during test execution.

- Black-Box Approach — System tested purely based on inputs and expected outputs without knowledge of internal implementation details.
- Module-wise Testing — Each module — Authentication, Onboarding, Assessment Engine, Skill Merging Engine, AI Roadmap Generation, Dashboard, and Projected Score — was tested independently before integration testing.

- Positive and Negative Testing — Both valid inputs (expected to succeed) and invalid inputs (expected to fail gracefully) were tested for every feature.
- Boundary Testing — Skill proficiency boundary values (exactly 40% and exactly 50%) were specifically tested to verify correct classification behaviour at threshold limits.
- Priority-Based Testing — Critical flows such as JWT authentication, skill merging classification, and AI roadmap generation were given highest priority and tested first.
- Re-Testing — After every bug fix, the affected module was re-tested to confirm the fix worked correctly and no new issues were introduced.
- End-to-End Testing — Complete user journeys were tested from registration through onboarding, role selection, MCQ assessment, and dashboard rendering to ensure smooth workflow across all modules.

6.2 TEST PLAN

The test plan defines the scope, approach, and schedule of all testing activities carried out for the system.

- **Scope** — All modules including Authentication, User Onboarding, Resume Upload, Role Selection, MCQ Assessment Engine, Intelligent Skill Merging Engine, Groq AI Roadmap Generation, Analytics Dashboard, and Projected Score Simulation.
- **Approach** — Manual black-box testing with predefined test cases for each module covering both positive and negative scenarios.
- **Test Environment** — Local machine running Node.js development server at http://localhost:5000 (backend) and Vite development server at http://localhost:5173 (frontend) with MongoDB running locally.

- **Test Data** — Demo user accounts with varying skill profiles, predefined JobRole profiles for Full Stack Developer, Data Scientist, and DevOps Engineer, and a seeded QuestionBank with 15-20 questions per role.

- **Entry Criteria** — All modules fully developed and integrated, application running without startup errors, MongoDB seeded with role profiles and question bank data, Groq API key configured in environment variables.

- **Exit Criteria** — All critical test cases passed, all identified defects resolved, no high-priority bugs remaining in the system, fallback roadmap verified to activate correctly on API failure.

6.3 TYPES OF TESTING

1. Unit Testing

Individual functions such as the Skill Merging Engine (skillMerger.js), per-skill proficiency calculation, match score computation, and career level determination were tested in isolation to verify they produce correct and consistent outputs for a variety of given inputs before being integrated into the full system. Boundary conditions at 40% and 50% proficiency thresholds were specifically tested to confirm correct classification behaviour.

2. Integration Testing

Modules were tested together to verify correct data flow between components. For example, when a user submits MCQ answers, the assessment engine must correctly evaluate responses, compute per-skill proficiency, pass the proficiency map to the Skill Merging Engine, trigger the Groq API for roadmap generation, store the resulting LearningPath in MongoDB, and return the complete dashboard data to the frontend — all in a single end-to-end flow.

3. Functional Testing

All features were tested against their defined functional requirements — registration with bcrypt password hashing, JWT login and protected route access, PDF resume upload and text extraction, role selection and required skills loading, MCQ question delivery without answer

field, server-side answer evaluation, five-category skill classification, match score calculation, AI roadmap generation, dashboard rendering, task checkbox tracking, and projected score modal activation.

4. Security Testing

Password hashing via bcrypt (10 salt rounds), JWT token generation and expiry enforcement, protected route middleware, and server-side answer storage (never transmitted to client) were all tested to ensure that unauthorized users cannot access protected endpoints or obtain MCQ answers through API inspection.

5. Usability Testing

The multi-step interface was tested for ease of navigation, clarity of form labels and error messages, correct step-locking behaviour (preventing step-skipping without completing mandatory fields), responsiveness of dashboard visualizations, and overall user experience across the complete five-step onboarding and assessment flow.

6. Negative Testing

Invalid inputs such as incorrect login credentials, expired JWT tokens, unrecognized job role names, empty skill arrays, malformed MCQ submission payloads, and direct access to protected API endpoints without authentication headers were all tested to verify proper error handling and appropriate HTTP status code responses (400, 401, 403, 404).

7. Fallback Testing

The Groq API timeout scenario was simulated to verify that the fallback roadmap generation module activates correctly and returns a valid rule-based roadmap without any system failure or 500 error response.

6.4 TEST CASES

TC No.	Input: Declared	Input: Test Score	Expected Category	Result
TC-01	React (declared)	React: 75%	Verified	Pass
TC-02	Not declared	CSS: 60%	Discovered	Pass
TC-03	Node.js (declared)	No data	Declared	Pass
TC-04	JavaScript (declared)	JS: 30%	Weak	Pass
TC-05	Not declared	No data	Missing	Pass
TC-06	Not declared	MongoDB: 35%	Weak (not Discovered)	Pass
TC-07	React (declared)	React: 50%	Verified (boundary)	Pass
TC-08	React (declared)	React: 40%	Weak (boundary)	Pass

Table No. 10: Unit Test Cases — Skill Merging Engine

6.5 VERIFICATION AND VALIDATION

Verification ensures that the system is built correctly — that the code correctly implements the design and requirements as specified during the planning phase. All four MongoDB collections (Users, JobRoles, QuestionBank, LearningPaths) were verified to match the designed schema with correct field names, data types, and referential integrity. All Express.js routes were verified to correctly handle both GET and POST requests and return appropriate HTTP status codes and response structures. The JWT middleware was verified to correctly block all protected endpoints when no token or an invalid token is provided. The answer field of QuestionBank documents was verified to be absent from all GET /api/assessment/:role responses, confirming server-side-only storage. The Skill Merging Engine was verified to produce consistent classification results across all five categories for all defined boundary and normal test inputs.

Validation ensures that the correct system is built — that the final product actually solves the real-world problem it was designed for. Users can successfully receive an objective, multi-signal skill analysis within minutes of completing the assessment without any technical knowledge required. The system correctly detects Discovered skills — competencies users possess but did not declare — demonstrating the value of the testing component over self-assessment alone. The AI-generated roadmap correctly prioritizes Weak skills before Missing skills in all tested scenarios. The Projected Score correctly reflects a value greater than the current Match Score when roadmap tasks are completed, confirming the simulation module produces logically valid results. The complete user journey — from registration through assessment to dashboard — works seamlessly without any data loss or state errors between steps.

6.6 TEST ENVIORNMENT

The testing of the AI-Based Skill Gap Analyzer and Roadmap Generator was carried out on a Windows 11 machine using Google Chrome (latest version) as the primary browser for all test case execution. The backend application was run locally on the Node.js/Express.js development server accessible at http://localhost:5000 with MongoDB running as a local instance storing data in the default data directory. The frontend React application was served by the Vite development server at http://localhost:5173. Node.js v18 LTS was used as the runtime environment throughout the entire testing process and all dependencies were installed via npm in isolated node_modules directories for both the frontend and backend to avoid any package conflicts.

The Groq API was accessed using a free-tier API key configured in the backend .env file, with a stable internet connection maintained during all AI roadmap generation test cases. API timeout simulation was performed by temporarily setting the Groq API request timeout to 1

millisecond to trigger the fallback roadmap module. A pre-seeded MongoDB database containing three JobRole profiles (Full Stack Developer, Data Scientist, DevOps Engineer) and their corresponding QuestionBank entries was used throughout testing to simulate a realistic environment without needing to manually recreate data before each test session. All tests were performed at a screen resolution of 1920x1080 to ensure proper UI rendering and dashboard layout verification across all sections.

6.7 TEST SUMMARY AND REPORTING

A total of 28 test cases were designed and executed during the testing phase of the AI-Based Skill Gap Analyzer and Roadmap Generator, covering all major modules including authentication, onboarding, resume upload, role selection, MCQ assessment, skill merging classification, AI roadmap generation, dashboard rendering, task tracking, projected score simulation, and security access control. The test cases were carefully designed to cover both positive scenarios — where valid inputs are expected to produce correct outputs — and negative scenarios — where invalid inputs are expected to produce appropriate error messages and graceful handling. All 28 test cases were executed successfully and passed without any failures, resulting in an overall pass rate of 100%.

During the testing process, a total of 5 defects were identified across different modules — MCQ answer exposure, boundary condition classification error, projected score button state synchronization, silent PDF upload failure, and fallback roadmap deduplication. All 5 defects were promptly analyzed, fixed, re-tested with the same test cases, and verified as resolved. The overall testing process confirmed that the AI-Based Skill Gap Analyzer and Roadmap Generator is stable, secure, and fully functional across all modules and user flows. No critical, high, or medium priority bugs remain in the final version of the application, and the system is fully ready for deployment, demonstration, and academic evaluation.

CHAPTER-6

SYSTEM IMPLEMENTATION

1 System Requirements — Acquisition

Category	Component	Specification
Hardware — Dev	Processor	Intel Core i5 / AMD Ryzen 5 or higher
Hardware — Dev	RAM	Minimum 8 GB (16 GB recommended)
Hardware — Dev	Storage	Minimum 10 GB free disk space
Hardware — Dev	Internet	Stable broadband connection (required for Groq API)
Software — Dev	Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+
Software — Dev	Node.js	v18.0 or higher (LTS recommended)
Software — Dev	Npm	v9.0 or higher
Software — Dev	MongoDB	v6.0+ (local) or MongoDB Atlas (cloud)
Software — Dev	IDE	VS Code with ESLint and Prettier extensions
Software — Dev	Browser	Google Chrome v110+ / Mozilla Firefox v110+
Software — Runtime	Groq API Key	Free-tier API key from console.groq.com
Software — Runtime	Environment Variables	.env file: MONGO_URI, JWT_SECRET, GROQ_API_KEY, PORT
Memory — Runtime	Backend Process	Approx. 150-300 MB RAM per Node.js instance
Storage — Runtime	MongoDB Data	Approx. 500 MB for full question bank and 10,000 user records

Table No. 11: System Hardware and Software Requirements

2 Changeover Procedure

Since this is a new system with no pre-existing digital counterpart in the target domain, a Direct Changeover approach is recommended for deployment. The system is made available to users immediately upon deployment without a parallel run of a legacy system. This approach is appropriate because:

- There is no existing automated system to run in parallel — the predecessor is entirely manual (self-assessment).
- The risk of system failure is mitigated by the fallback roadmap mechanism, which ensures the system remains functional even if the Groq API is unavailable.
- The system can be rolled back to the previous version by reverting to the prior deployment commit on the hosting platform.

For production deployment, the following procedure is recommended:

- Deploy the MongoDB instance (MongoDB Atlas recommended for production).
- Seed the database with initial JobRole profiles and QuestionBank entries using the provided seed scripts.
- Deploy the Node.js backend to a cloud platform (e.g., Render, Railway, or AWS EC2).
- Build the React frontend (npm run build) and deploy the dist/ folder to a static hosting service (e.g., Vercel or Netlify).
- Configure the frontend Axios base URL to point to the deployed backend URL.
- Perform smoke testing on the live environment before user-facing launch.

3 Training Needs

The system is designed to be fully self-explanatory through its guided multi-step flow and contextual UI labels. However, the following training considerations apply:

3.1 End Users (Students and Job Seekers)

No formal training is required. The multi-step flow guides users through each stage with clear instructions and input labels. The dashboard uses visual elements (color-coded badges, progress bars, timeline) to present complex data accessibly. A brief onboarding tooltip system can be added in a future iteration to explain each dashboard section.

3.2 Administrators

System administrators responsible for maintaining the question bank and job role profiles require familiarity with MongoDB Compass or the MongoDB Atlas interface for direct database management. A future admin panel UI is planned to allow non-technical administrators to add and edit job roles and questions without database access.

3.3 Developers

New developers onboarding to the project should review the README.md file (covering setup, environment configuration, and API documentation), the database schema definitions in the /models directory, and the Intelligent Skill Merging Engine logic in /services/skillMerger.js. The modular project structure ensures that individual components can be understood in isolation.

4 Documentation

4.1 Operation Manual

To run the system locally:

- Clone the repository from GitHub.
- Navigate to the /backend directory and run: npm install
- Seed the database: node seed/seedRoles.js && node seed/seedQuestions.js
- Start the backend server: npm start (or npm run dev for development with nodemon)

- Navigate to the /frontend directory and run: npm install
- Start the frontend: npm run dev
- Access the application at http://localhost:5173 in a web browser.

4.2 User Manual

For end users:

- Open the application URL in a web browser.
- Click 'Register' to create a new account. Enter your name, email, and a password. Click 'Create Account'.
- On the onboarding page, enter your current skills separated by commas. Optionally upload your resume as a PDF file.
- On the role selection page, click the card for your target job role.
- Complete the MCQ assessment by selecting one answer per question. Click 'Submit Assessment' when finished.
- Review your personalized analytics dashboard. The Match Score circle shows your current skill coverage. The Skill Breakdown section shows the status of each skill.

SUMMARY/CONCLUSION

This project presents the design and implementation of an AI-Based Skill Gap Analyzer and Roadmap Generator — a full-stack web application that addresses the critical limitations of self-assessment-based career guidance. The system integrates user authentication, multi-step onboarding, resume upload, role-specific MCQ assessment, an Intelligent Skill Merging Engine, Groq AI-powered roadmap generation, and an interactive analytics dashboard with a Projected Score Simulation module into a single, seamless platform.

The system was developed using the Agile SDLC model across eight sprints, producing a functional, tested, and documented application. The technology stack — React.js (Vite), Node.js/Express.js, MongoDB, and the Groq API (LLaMA 3.3-70B) — was selected for its maturity, performance, and cost-effectiveness in an academic context.

The AI-Based Skill Gap Analyzer and Roadmap Generator successfully achieves all eight project objectives defined in Chapter 1. The core innovation — the Intelligent Skill Merging Engine — demonstrates that combining declared skills with objective test performance produces a richer, more accurate competency profile than either signal alone. The detection of Discovered skills addresses a real gap in existing tools that rely exclusively on self-reported data.

The separation of Match Score (coverage) and Test Score (depth) avoids the misleading aggregation common in platforms that conflate these distinct dimensions of skill proficiency. The Projected Score Simulation transforms the learning roadmap from a static list into an interactive, motivating tool that quantifies the benefit of learning investment.

This project makes a meaningful contribution to the domain of AI-driven career development tools and provides a strong foundation for future enhancement and production deployment.

LIMITATIONS OF THE PROJECT

1. **Limited Resume Parsing** - The resume parser currently performs only plain-text extraction and does not use advanced NLP-based entity recognition techniques to accurately identify skills, certifications, technologies, or experience from unstructured resume content.

2. **Static Question Bank** - The assessment question bank is manually seeded and supports only a limited number of predefined job roles. Adding support for new domains or roles requires manual question creation and database updates.

3. **No Adaptive Difficulty System** - The assessment process does not dynamically adjust question difficulty based on user performance. All candidates attempting the same role receive a fixed set of questions regardless of their skill level or previous answers.

4. **No Longitudinal Progress Tracking** - The system generates a new LearningPath record for every assessment session without maintaining historical linkage between sessions, making it difficult to analyze long-term user improvement and performance trends.

5. **External AI Dependency** - The platform relies heavily on the Groq API for AI-generated analysis and recommendations. Although a fallback mechanism exists, the fallback system is rule-based and produces less accurate and less context-aware results compared to AI-generated outputs.

6. **No Multi-Language Support** - The application currently supports only the English language for both the user interface and generated outputs, which limits accessibility for users from non-English-speaking regions.

FUTURE DIRECTIONS

- **Semantic Resume Parsing:** Integrate NLP-based named entity recognition (e.g., spaCy or Hugging Face token classification models) to automatically extract and normalize skill entities from unstructured resume text.
- **LinkedIn API Integration:** Connect with LinkedIn's API to import professional experience, endorsements, and current job market skill demand data, enriching both the skill declaration baseline and the job role requirement profiles.
- **Longitudinal Progress Tracking:** Implement a tracking module that links multiple assessment sessions per user, enabling skill development trajectory visualization over weeks and months.
- **Adaptive Assessment (IRT):** Implement Item Response Theory-based adaptive MCQ delivery where question difficulty adjusts in real-time based on the user's running performance during the assessment, improving measurement accuracy.
- **Peer Comparison and Leaderboard:** Add anonymized benchmarking features enabling users to compare their match scores and skill profiles against others targeting the same role.
- **Admin Panel:** Develop a web-based admin UI for non-technical administrators to manage job role profiles and question bank entries without direct database access.
- **Multi-Language Support:** Extend the interface, question bank, and AI prompts to support multiple languages for broader accessibility.
- **Mobile Application:** Develop native iOS and Android applications with push notification-based learning reminders.

REFERENCES / BIBLIOGRAPHY

[1] Khanal, S. S., Prasad, P. W. C., Alsadoon, A., & Maag, A. (2020). A systematic review: machine learning based recommendation systems for e-learning. Education and Information Tchnologies, 25, 2635-2664.

[2] Chatti, M. A., Dyckhoff, A. L., Schroeder, U., & Thus, H. (2012). A reference model for learning analytics. International Journal of Technology Enhanced Learning, 4(5-6), 318-331.

[3] Xu, J., Xing, T., & Van der Schaar, M. (2016). Personalized course sequence recmendations. IEEE Transactions on Signal Processing, 64(20), 5340-5352.

[4] Aher, S. B., & Lobo, L. M. R. J. (2013). Combination of machine learning algorithms for recommendation of courses in E-Learning System based on historical data. Educational Data Mining, 5(4), 37-55.

[5] Imran, A. S., Dalipi, F., & Folke, M. (2021). AI-Powered Adaptive Learning Systems. IEEE Access, 9, 98345-98360.

[6] Romero, C., & Ventura, S. (2020). Educational data mining and learning analytics: An updated survey. WIREs Data Mining and Knowledge Discovery, 10(3), e1355.

[7] Baker, R. S., & Inventado, P. S. (2014). Educational data mining and learning analytics. In Larusson J., White B. (eds), Learning Analytics. Springer.

[8] Fazeli, S., Drachsler, H., & Sloep, P. (2017). Applying recommender systems for learning analytics: A tutorial and review. New Directions for Teaching and Learning, 2017(152), 105-124.

[9] Verbert, K., Manouselis, N., Ochoa, X., Wolpers, M., Drachsler, H., Bosnic, I., & Duval, E. (2012). Context-aware recommender systems for learning: A survey and future challenges. IEEE Transactions on Learning Technologies, 5(4), 318-335.

[10] Dillahunt, T. R., Wang, X., & Teasley, S. D. (2016). Democratizing Higher Education: Exploring MOOC Use Among Those Who Cannot Afford a College Education. International Review of Research in Open and Distributed Learning, 17(6).

[11] Groq Inc. (2024). Groq API Documentation. Retrieved from https://console.groq.com/docs

[12] Meta AI. (2024). LLaMA 3.3 Model Card. Retrieved from https://ai.meta.com/llama

[13] MongoDB Inc. (2024). MongoDB Documentation: Schema Design Best Practices. Retrieved from https://www.mongodb.com/docs

[14] Facebook Inc. (2024). React Documentation. Retrieved from https://react.dev

[15] Node.js Foundation. (2024). Node.js Documentation. Retrieved from https://nodejs.org/en/docs

[16] Express.js Contributors. (2024). Express Documentation. Retrieved from https://expressjs.com

[17] Mongoose Contributors. (2024). Mongoose ODM Documentation. Retrieved from https://mongoosejs.com/docs

[18] JSON Web Tokens. (2024). JWT Introduction. Retrieved from https://jwt.io/introduction

[19] npm. (2024). bcrypt package documentation. Retrieved from https://www.npmjs.com/package/bcrypt

[20] npm. (2024). pdf-parse package documentation. Retrieved from https://www.npmjs.com/package/pdf-parse